



TOPICAL REVIEW • **OPEN ACCESS**

Outlook towards deployable continual learning for particle accelerators

To cite this article: Kishansingh Rajput *et al* 2025 *Mach. Learn.: Sci. Technol.* **6** 031001

View the [article online](#) for updates and enhancements.

You may also like

- [Bridging text and crystal structures: literature-driven contrastive learning for materials science](#)
Yuta Suzuki, Tatsunori Tanai, Ryo Igarashi et al.
- [Mamba time series forecasting with uncertainty quantification](#)
Pedro Pessoa, Paul Campitelli, Douglas P Shepherd et al.
- [Ordered embeddings and intrinsic dimensionalities with information-ordered bottlenecks](#)
Matthew Ho, Xiaosheng Zhao and Benjamin D Wandelt



TOPICAL REVIEW

OPEN ACCESS

RECEIVED
10 April 2025REVISED
6 June 2025ACCEPTED FOR PUBLICATION
2 July 2025PUBLISHED
11 July 2025

Original Content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Outlook towards deployable continual learning for particle accelerators

Kishansingh Rajput^{1,2,*} , Sen Lin², Auralee Edelen³, Willem Blokland⁴ and Malachi Schram¹ ¹ Thomas Jefferson National Accelerator Facility, Newport News, VA 23606, United States of America² Department of Computer Science, University of Houston, Houston, TX 77204, United States of America³ SLAC National Laboratory, Menlo Park, CA 94025, United States of America⁴ Spallation Neutron Source, Oak Ridge National Laboratory, Oak Ridge, TN, United States of America

* Author to whom any correspondence should be addressed.

E-mail: kishan@jlab.org**Keywords:** drift, continual learning, adaptive machine learning, particle accelerator, distribution shift, adaptive model, deployable

Abstract

Particle accelerators are high power complex machines. To ensure uninterrupted operation of these machines, thousands of pieces of equipment need to be synchronized, which requires addressing many challenges including design, optimization and control, anomaly detection and machine protection. With recent advancements, machine learning (ML) holds promise to assist in more advance prognostics, optimization, and control. While ML based solutions have been developed for several applications in particle accelerators, only few have reached deployment and even fewer to long term usage, due to particle accelerator data distribution drifts caused by changes in both measurable and non-measurable parameters. In this paper, we identify some of the key areas within particle accelerators where continual learning can allow maintenance of ML model performance with distribution drifts. Particularly, we first discuss existing applications of ML in particle accelerators, and their limitations due to distribution drift. Next, we review existing continual learning techniques and investigate their potential applications to address data distribution drifts in accelerators. By identifying the opportunities and challenges in applying continual learning, this paper seeks to open up the new field and inspire more research efforts towards deployable continual learning for particle accelerators.

Glossary

A-GEM	Average gradient episodic memory.
ADWIN	Adaptive windowing.
AE	Auto-encoder.
AI	Artificial intelligence.
ANN	Artificial neural network.
AWAKE	Advanced proton driven plasma wakefield acceleration experiment.
BNL	Brookhaven National Laboratory.
BO	Bayesian optimization.
BPM	Beam position monitor.
CEBAF	Continuous electron beam accelerator facility.
CERN	European Organization for Nuclear Research.
CPU	Central processing unit.
DCM	Differential current monitor.
DMPC	Deep model predictive control.
DRL	Deep reinforcement learning.
DT	Digital twin.
EWC	Elastic weight consolidation.
FNAL	Fermi National Accelerator Laboratory.
FRIBs	Facility for rare isotope beams.

GAN	Generative adversarial network.
GEM	Gradient episodic memory.
GP	Gaussian process.
GPU	Graphics processing unit.
HPC	High performance computing.
HVCM	High voltage converter modulator.
KARA	Karlsruhe Research Accelerator.
LBNL	Lawrence Berkeley National Laboratory.
LCLS	Linac coherent light source.
LMU	Ludwig-Maximilians-Universität München.
LSTM	Long short term memory.
LwF	Learning without forgetting.
MAML	Model-agnostic meta-learning.
MASs	Memory aware synapses.
MEBT	Medium energy beam transport.
ML	Machine learning.
MORL	Multi-objective reinforcement learning.
MPC	Model predictive control.
NERSC	National Energy Research Scientific Computing Center.
NLP	Natural language processing.
NMN	Neural modular network.
OGD	Orthogonal gradient descent.
OOD	Out-of-distribution.
PNN	Progressive neural network.
RF	Radio frequency.
RL	Reinforcement learning.
RNN	Recurrent neural network.
SCL	Super conducting linac.
SLAC	SLAC National Laboratory.
SNN	Siamese neural network.
SNS	Spallation Neutron Source.
VAE	Variational auto-encoder.
XFEL	X-ray free-electron laser.

1. Introduction

Particle accelerators [1] are of great importance in the investigation of the fundamental building blocks of matter and advancing our understanding of the Universe with many facilities globally operating them to conduct science experiments. Smaller scale particle accelerators are also leveraged in many industrial applications, including sanitation, medical imaging and cancer therapy [2]. To maintain optimal performance, these accelerator systems require meticulous tuning and adjustment. Typically, operators utilize either manual fine-tuning by adjusting many precision knobs or leverage sophisticated automated optimization algorithms to achieve precise control.

This is very challenging due to the large number of adjustable parameters and the sensitivity of the beam responses. As such, operators spend significant effort and time tuning the accelerator beam at the beginning of a run, especially if the accelerator was down for a long time or any upgrades to the equipment had been performed. In some cases there is a lack of appropriate physics models to capture the behavior of the accelerated beam, such as beam halo losses [3]. A ‘beam halo’ refers to the region around the desired particle beam where particles are being deflected or scattered due to various sources of radiation and interactions with surrounding materials. Heuristic approaches, e.g. operators manually adjusting the accelerator settings to minimize losses, are used which often require significant time to complete the optimization. These challenges can be exacerbated if an accelerator needs to frequently switch among different types of beams or beam power requiring different settings for the equipment. For example, the FRIBs uses different beam species for different experiments. Adjustments to control algorithms must be made and/or the heuristic knowledge must be learned for the new setup.

During operation, the accelerator beam can easily stray from the intended path when any equipment deviates from its expected behavior, resulting in radio-activation or worse, damage, unless the beam is quickly aborted. This is a problem especially for high-power beams, where even stray halo can cause damage. While preventative maintenance is scheduled and many diagnostics sensors are placed along the beam line to detect these errant beam conditions and protect the equipment from damage by aborting the beam, unexpected downtime due to faults still occur.

Additional complexity arises from the tendency of accelerators to experience both gradual and systematic change in behavior or characteristics over time during operation due to various reasons including machine aging, and environmental factors. This phenomenon, referred as ‘drift’, can significantly impact performance and requires adjustments in accelerator settings. As such, there are challenges with respect to: (1) minimizing time spent on tuning for different setups to maximize time for experiments, (2) optimizing performance in the absence of physics models, and (3) detecting and minimizing the impact of abnormal or drifting conditions.

With the rapid increase in the storage and computational power, ML has achieved astonishing successes in addressing complex problems in science and engineering, including material science [4], bio-informatics [5], radiology [6], renewable energy [7], physics [8] and many other fields. In particular, ML-based methods can outperform many traditional methods by leveraging ANN [9] for feature learning. Thus motivated, there have emerged a lot of studies to develop ML-based solutions for various applications in particle accelerators, including anomaly detection [10], preemptive anomaly prediction with uncertainty quantification [11], optimization [12], dynamic control [13], and surrogate models [14]. ML techniques have also been adopted in real-time decision support [15] and control applications [16] for more accurate and faster turn-around.

Nevertheless, the evolving nature of real-world systems poses a significant challenge for current ML approaches being directly used in practice: standard ML assumes a stationary distribution for the data, whereas the data distribution and the underlying concept can dynamically change with time in real systems. In dynamic machines such as particle accelerators, both data and the input to target relationship drift over time due to various reasons including but not limited to environmental changes, and machine degradation [17], which may lead to a substantial performance drop of a deployed ML model [18]. Therefore, a critical requirement towards deployable ML is the model robustness in predictions, especially against inherent noise in accelerator measurements, occasional failures of signals (e.g. a read-back failure), and both deliberate and unintended time-varying changes in conditions. These can result in new data patterns outside of the statistical distribution of the training data [18], commonly known as concept drift [19], which is difficult for ML models to handle due to their limited capacity to extrapolate beyond the scope of the training data distribution [20]. In particular, concept drift is more evident in particle accelerators due to continuous stream of drifting data, and developing robust learning systems is essential to carry out the practical benefits of ML in this domain.

Continual learning [21], also known as lifelong learning, addresses performance degradation with continual model updates under data distribution drifts. It provides a promising solution for maintaining ML models’ practical performance in a dynamic environment. Note that continual learning is distinct from online incremental learning. The latter enables adaptation to new data as it arrives and does not include any mechanism to retain previous knowledge, whereas the former also considered retention of previously learned knowledge. Notably, the importance of continual learning is underscored by its applicability across various domains, ranging from computer vision [22] and natural language processing [23] to robotics and autonomous systems [24]. For instance, in computer vision, continual learning is crucial for developing systems capable of adapting to changing visual contexts and learning new object categories without forgetting previously learned ones. Similarly, in the field of robotics, continual learning enables robots to acquire new skills and adapt to diverse environments over their operational lifespan.

Despite the great potentials of continual learning, its applications in particle accelerators has not been fully explored and there still lacks a dedicated avenue for exchanging the underlying ideas. To fill this void, this paper seeks to present a review of the challenges and opportunities in continual learning as it applies to particle accelerators. We critically evaluate existing continual learning methods, assessing their relevance and applicability within the particle accelerator domain. We describe potential future avenues for research in continual learning, providing concrete applications that can propel advancements for ML in particle accelerators. Our analysis aims to identify key challenges and opportunities in application of continual learning in accelerators, thereby paving the way for future advancements and applications in this domain. This paper mainly focuses on applications of ML for anomaly prediction/detection, optimization, real-time control, surrogate models and DTs, and virtual diagnostics to provide valuable insights to adapt ML models applied in these use-cases with continuously drifting accelerator data.

The remainder of the paper is organized as follows. Section 2 provides a brief background on existing applications of ML in particle accelerators, and their limitations, section 3 describes challenges associated with deployment of ML solutions in particle accelerators, section 4 presents a short but comprehensive review on continual learning and the promises it holds. Section 5 presents potential areas within particle accelerators where continual learning methods can help mitigate the deployment challenges associated with data drift. We also highlight the feasibility of different continual learning methods for various applications given constraints linked to individual applications. We close the discussion with a conclusion and future outlook in section 6.

2. AI/ML in particle accelerators

Many areas within particle accelerators have shown remarkable improvement in both performance and time-efficiency with assistance from ML, including anomaly detection and prognostication to avoid downtime and equipment damages, optimization of design, optimization of operation and real time control, beam tuning and shaping etc. However, the potential of these solutions has not been fully realized in operational usage due to challenges associated with data drifts, ML lifecycle management and time sensitivity. In what follows, we first review some of the recent developments at the intersection of ML and particle accelerators, and describe how they fit in the context of continual learning.

2.1. Anomaly detection

AEs: Particle accelerators produce a large volume of data recorded by various diagnostics and monitoring sensors such as BPM and DCM. A majority of this data is from normal operation and a small amount of data represent anomalies. Normal machine behavior can also be mimicked by simulations. This leads to a natural class imbalance for anomaly detection applications. AE [25] or VAE [26] are unsupervised (or semi-supervised in some cases) ML algorithms that can be trained on imbalanced datasets without labels, by leveraging large amount of existing normal data and clustering the latent space to detect anomalies. The underlying assumption is that the similarity between normal samples would lead their latent representations to be close together, whereas latent representation of anomaly samples would be far away from the normal cluster(s). In this direction, a resilient VAE model has been developed to detect anomalies at LCLS at SLAC and shown to perform well when trained on a large volume of normal data and a small set of anomaly data [27]. On the other hand, AE/VAE can also be trained only on normal data (semi-supervised) to capture the features that build the normal data samples and reconstruct them with a very low error. During inference, it is expected that the model would have high reconstruction error on anomaly samples as they are expected to be different from the training data. Here, instead of leveraging latent representation, reconstruction error is used as a classification indicator to detect anomalies. One such model was developed at BNL to detect anomalies in air conditioning system of their accelerator [28]. In a similar effort, semi-supervised AE-based ML model is developed for real-time anomaly detection in the CMS electromagnetic calorimeter at CERN [29]. This model had shown great potential during its initial deployment but long term online deployment requires drift handling. In addition, a modified VAE based model demonstrates strong potential for implementation as a hardware-level anomaly trigger in collider experiments being tested on simulated Standard Model and Beyond Standard Model processes at CERN [30].

Particle accelerators are actively tuned and acquire multiple different configuration settings during normal operation. They also have multiple sub-systems producing similar data, where the normal data collected from different sub-systems or different run conditions can vary producing multiple data distributions. In such cases, conditional VAE is leveraged at the SNS accelerator for errant beam prediction in SCL [31]. In a similar effort, conditional VAE is leveraged to learn the distribution of the data originating from different HVCM to detect anomalies [32]. Additionally, a study at SNS accelerator has shown that contrastive learning performs much better at anomaly prediction than reconstruction based methods such as AE [31]. These applications have been demonstrated on using curated offline dataset or during short online studies where conditions are relatively constant. Deploying these solutions for longer term online applications would require continual model adaptation as AE/VAE can lead to higher reconstruction error or misleading clustering on drifted data.

Contrastive learning: contrastive learning has been mainly developed for few shot learning in ML [33] where the feature extraction knowledge can be transferred from one dataset to another before fine-tuning with a minimal number of samples (few shots) from the new dataset. One intrinsic property of contrastive learning methods, such as SNN [34], is that it can also address class imbalance in the training data. Specifically, SNN requires a pair of inputs and learns to predict similarities between the two input samples. By exploiting this property, a large amount of normal data can be paired with a small number of anomaly samples to produce the positive class, and a roughly equal number of pairs can be created with normal–normal data as negative class samples. Recent studies at SNS accelerator have shown that SNN can outperform semi-supervised AE/VAE [31]. The study was further improved by developing conditional SNN which improves the predictions of errant beams at SNS when considering multiple beam setting parameters. However, the study also identified a need for continual learning to maintain model performance on time varying data drifts [31].

Other techniques for anomaly detection on accelerators include supervised classification based feed-forward neural network, explored in pressure systems at SuperKEKB accelerator [35]. For time-series data, RNN [36] models have been developed to detect anomalies in RF cavities at CEBAF [37]. Another effort at SNS studied

detection of anomalies in HVCM system using LSTM networks [38]. These applications use neural network that are shown to have performance degradation on drifting data [18], as such continual learning can enable their long term usage.

2.2. Optimization and control

Particle accelerators require active tuning of various equipment before and during operation. Human operators are often tuning these machines for hours to achieve required beam delivery. ML based optimization and control algorithms has been explored to help alleviate these challenges [39]. Various algorithms may incorporate models that persist across multiple tuning sessions, thereby presenting a topic of interest from the perspective of continual learning. For example, with standard BO, a new model is learned from scratch during each optimization run. In advanced BO implementations, it leverages priors from learned system models, which may need to be updated over time. This is similar to MPC algorithm that often uses a separately-developed system model that persists along with a policy model.

One application of model updating in online control for accelerators is presented in [40], where a model-based approach is employed to implement feed-forward correction for regulating beam sizes in the presence of varying insertion devices. In this framework, a base model is trained on a diverse set of data distributions. When encountering a new data distribution, the same base model undergoes fine-tuning to adapt to the updated conditions. This methodology aligns with principles of online learning. However, it does not address the issue of catastrophic forgetting [41], as the specific application does not necessitate retaining prior knowledge. Instead, the model is only required to reflect the most current state of the machine.

BO BO [42] is a powerful framework designed to optimize black-box models, particularly in the context of system parameters such as magnet settings in accelerators. The algorithm intends to find optimal settings by effectively mapping input variables to a target objective function. Utilizing GP [42], BO not only estimates the function values but also quantifies uncertainties associated with predictions. One of the significant advantages of BO is its ability to operate with limited data, enabling the development of a local system model from the ground up. This local model enhances the efficiency of the optimization process, allowing BO to learn from fewer samples compared to traditional methods. Additional information about uncontrolled inputs can be incorporated into ‘contextual’ BO, but this is limited by the computational scaling of GPs. BO also enables learned output constraints to easily be incorporated into tuning, which is appealing for deployment on unexplored systems or setups [39]. BO has been used to optimize free-electron laser at SLAC [43]. In addition, BO has been explored for optimization applications such as beam injection process into a storage ring at KARA [44], and at cooler synchrotron COSY [45], for recoil mass separator [46], and accelerator tuning [47]. A safe BO variant has been applied to tune Proportional Integral controllers in the optical synchronization system of the European XFEL, aiming to improve experimental precision while minimizing costly machine time [48]. In addition, BO has been explored for SASE pulse energy tuning at the European XFEL, demonstrating performance comparable to the traditional simplex method [49] without requiring manual hyperparameter tuning [50]. The results highlight BO’s potential as a low-effort, adaptable tool for routine accelerator optimization, with further improvements possible through integration of physical priors. At LMU, BO has been explored for multi-objective optimization [51] and pareto learning to balance trade-offs between multiple objectives [52].

Fast and accurate system models can be used to provide prior information, correlated and learned kernels for BO to improve learning and increase convergence speed during online tuning [53]. Continual learning could be highly relevant for keeping system models used as priors, physics-informed kernels, and similar learning boosts up to date with data drifts.

RL DRL has emerged as a powerful approach in various control applications, particularly in robotics [54]. Its inherent alignment with time- and sequence- dependent control problems makes it well suited for identifying optimal control actions while considering the future evolution of the system. Although DRL typically requires more development overhead, it offers the advantage of continuous control capabilities. Once trained, DRL can provide solutions more rapidly than BO, and can also ingest large amounts of information from previous machine interactions or archived data. This allows DRL to leverage previously-learned correlations across different machine operating conditions, enhancing control performance.

DRL has shown promising results for both optimization and control applications within particle accelerators. Notable applications include its use for optimizing heat load and trip rates at Jefferson Lab CEBAF [55], and for booster control at Fermilab [16]. Researchers have illustrated the potential of RL algorithms to enhance the efficiency of particle accelerator operation, with optimization of beam injection at COSY facility in Germany [56], optimization and control at FERMI FEL facility in Italy [57], proof of concept autonomous sub-system control [58], beam line control [59], and multi-parameter control

tasks [60] at CERN accelerators, showing improved efficiency over traditional optimizers by learning optimal policies without repeated exploration with successful tests at AWAKE and LINAC4. While the use of MORL has been limited in the particle accelerator domain, it can naturally provide dynamic control capabilities. MORL has been evaluated for simultaneous optimization of heat load and trip rates in CEBAF linacs at Jefferson Lab [55]. A recent study on a multi-objective constrained optimization problem concluded that differentiable RL can outperform traditional MORL on large dimensional problems [12].

One of the key limitations of traditional DRL approaches in these applications is their inability to adapt efficiently to evolving machine conditions without extensive retraining or catastrophic forgetting. This is also evident from the fact that most of the above discussed DRL based solutions are yet to be seen as a long term deployed solution. Traditional DRL typically relies on continuous training with experience replay, where past experiences are stored and reused to stabilize learning. However, this approach is limited in dynamic environments, as it does not inherently adapt to evolving conditions and can suffer from catastrophic forgetting when faced with new operational scenarios. In contrast, continual learning enables DRL controllers to retain past knowledge by leveraging adaptive techniques including but not limited to memory based replays, gradient projections, and policy (and critic if applicable) neural network weights regularization while integrating new information allowing for real-time adaptation to drifting data. These advance aids to continual DRL makes it different from traditional DRL.

MPC Similar to DRL, DMPC is also suitable for time- and sequence- dependent control problems, where the goal is to find optimal control actions by considering the future evolution of the system. To achieve this, DMPC typically requires a sufficiently accurate and fast-executing system model. This approach enables additional constraining of control behavior and provides more directly interpretable actions. Moreover, DMPC has been successfully applied to various applications in particle accelerator space where system model(s) are available. Some of the notable applications include power supply control [61], data driven DMPC to refine the multi-turn injection process into the heavy ion-synchrotron SIS18 using Gaussian process model [62], application of DMPC on inner triplet heat exchanger unit of the large hadron collider at CERN based on a nonlinear model derived from physical relationships [63], ion-source control for beam tuning at RFT-30 cyclotron [64], and RF Quadrupole resonant frequency control in proton improvement plan II at FNAL [65]. The type of system model used in DMPC can vary between empirically tuned physics models, to learned neural network models, to Gaussian processes.

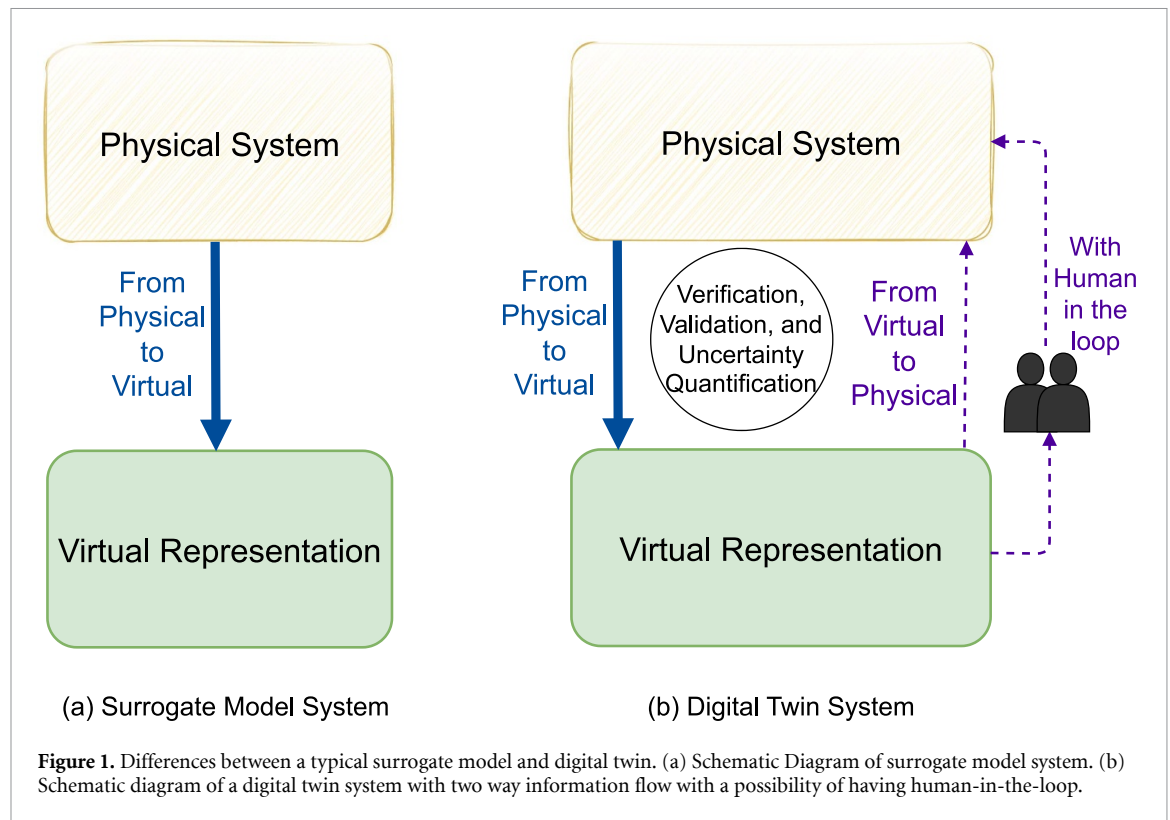
Continual learning can enhance the adaptability of neural network policy models with changing conditions. In addition, it is also applicable where neural networks are used as a system model and can provide great benefits to keep it up-to-date with drifting data.

2.3. Surrogate models and DT

A surrogate model is a computational model that is used to approximate the behavior of a more complex, detailed, and often computationally expensive process. Surrogate models are often used to replace physical processes that are resource-intensive, time consuming and/or expensive. On the other hand, a DT [66] provides a real-time, dynamic representation of a physical entity, process, or system. It integrates real-time data with simulation models to monitor, predict, and optimize the performance of the physical counterpart. As demonstrated in figure 1, surrogate models often have a one-way information flow where a data driven model is trained on measurements obtained from the physical machine. In contrast, a DT has a two-way information flow, where not only the virtual model is continuously adapted to mimic real-time physical system behavior but it is also used for decision making to improve the physical system. This process often may involve human operators in the loop.

In particle accelerators, both data and physics driven models have been developed and used for various purposes such as training of a control algorithm, diagnostics and pre-emptive maintenance. Simulating particle accelerator processes in digital space is both computationally expensive and time consuming. There are a number of physics based beam dynamics simulations including general particle tracer [67], virtual accelerator [68], Bmad [69], and pyOrbit [70], however, discussion of purely physics driven simulations is out of the scope of this paper.

To reduce computational resources requirements and execution time, many particle accelerator processes are being represented as data driven surrogate models [71]. These models have mainly been used to train control algorithms which are infeasible to train on the real physical machine due to various constraints. For example, a robust DT was developed at Fermi FNAL to train a RL agent [72]. A surrogate model is also used at ISIS Neutron and Muon Source to assist in tuning of beam losses [73]. Neural network based surrogate models of RF quadrupole [74], and an encoder-decoder neural network based surrogate model of MEBT [75] have been shown to be effective in simulations. A surrogate model is developed for a subset of the proposed compact linear collider final-focus system at CERN [76]. In addition, uncertainty aware surrogate



models have been studied at FNAL for their booster complex and shown good performance on detecting out of distribution samples [77].

Similarly, DT can be used to test accelerator physics software such as beam simulations or control systems before applying in the field. Several DTs have been developed in particle accelerator space, for example, DTs of KARA [78], and SLAC beamline [79]. Developing reliable continual learning approaches for DTs in accelerators is a major need, especially for systems which experience substantial drift over time or frequently change experimental setups.

2.4. Virtual diagnostics

Virtual diagnostics provide predictions of machine behavior that cannot be directly measured (at all or continuously). For example, the x-band transverse deflecting mode CAVity at FACET-II, and beam viewers at Upgraded Injector Test Facility at Jefferson Lab are destructive to the beam, and thus beam cannot be delivered to users and monitored on these sensors simultaneously. Virtual diagnostics can use other measured data to predict the output of the unavailable diagnostic.

Virtual diagnostics can use supervised learning to map measured inputs to predicted outputs (e.g. longitudinal phase space prediction [80], prediction of x-ray pulse properties from a free-electron laser [81], and prediction of photon beam properties such as pointing and divergence at the European XFEL's SASE1 beamline, using GENESIS simulations of various lasing conditions [82]). Other approaches adjust a learned system model that is adjusted based on observations to then provide a quasi-real-time estimate of other beam properties which are either inaccessible or require destructive measurements (e.g. see [83]). Data distribution drift over time is a major challenge for practical application of virtual diagnostics in practice. Users must have assurance that the predictions are going to continue to be accurate for use in analysis and control.

Evident from the above discussion, with development of many ML models for particle accelerators, researchers have shown that ML holds tremendous potential to enable autonomous operation of particle accelerators. However, it is evident that continual learning is critical to enable long term deployment of ML in particle accelerators and there are still several challenges associated with it as described next.

3. Deployment challenges in the context of continual learning

Despite significant advancements in AI and ML for particle accelerators, the number of implemented approaches remains notably behind the apparent progress. This gap arises from both algorithmic and practical challenges: some cases require fundamental improvements to AI/ML techniques to ensure reliable

deployment, while others involve mature techniques that still need the appropriate software infrastructure for effective implementation and integration into operations.

A significant technical challenge is *data distribution drifts* due to changes in the equipment settings, drift in the data due to unmeasured/unknown parameters such as machine degradation or replacement of equipment, and changes in the environmental conditions. The presence of data distribution drift means an ML solution cannot simply be created once and deployed; it needs to be monitored and updated over time. Addressing data distribution drift effectively requires both algorithmic improvements and infrastructure for continual learning. Other notable challenges include low latency requirements, large volume of data, large number of variables, and lack of appropriate infrastructure as described below.

3.1. Data and concept drift in particle accelerators

Data drift refers to changes in the statistical properties of input data, meaning that the features or distributions of the input data differ from those encountered during ML model training [84]. In complex machines, such as particle accelerators, any deviation in behavior of one or more pieces of equipment can quickly propagate and drift the distribution of the measurements downstream. Variations in initial conditions, changes in settings for optimization, and environmental factors can lead to drifts in the data that ML models were initially trained on. These drifts can adversely affect model performance, resulting in decreased accuracy and reliability in predictions. This is due to concept drift as the relationship between input features and target variables changes over time. As such, if the data distribution drifts, the ML model may extrapolate incorrectly or may not represent the up-to-date concept, leading to reduced accuracy and performance. Detecting and addressing concept drift is crucial to maintaining the reliability of ML outputs prompting the development of adaptive techniques such as retraining, ensemble methods, and drift detection algorithms [85]. Continuous monitoring, along with adaptive algorithms and retraining strategies, is essential to identify and mitigate the effects of data- and concept-drift, ensuring that ML models remain robust and effective in interpreting evolving experimental data. Data distribution drifts in particle accelerators can broadly be classified into the following two categories.

3.1.1. Data distribution drifts due to measured parameters

When the beam parameters and/or equipment settings are changed, they drift the measurements. These types of data distribution drifts are usually easier to detect and even predict in some cases as the changes in beam setting knobs can be quantitatively measured. However, these changes can still make an ML model obsolete if the drifts are significant. One of the ways to minimize such effects is to use conditional ML models where the beam settings and other relevant parameters are used as conditional inputs to the model(s) [31]. This helps the model learn the correlation between the data samples from different settings and effectively interpolate to new settings. However, if the settings go out of distribution with significant drift from the training data distribution, the model can still fail and requires to be fine-tuned.

3.1.2. Drifts due to non-measured parameters

Frequently particle accelerator data drifts due to unknown causes or if known relevant information is not recorded or non-quantifiable. For example, particle accelerator data distribution typically drifts after a long shut down or a maintenance period due to changes in the equipment. Slow (and some rapid) drifts are still observed during continuous operation of particle accelerators. This type of drifts can arise due to equipment degradation, drifts in ion source output as plasma changes (depleting source), variation in the performance of klystron over time, high voltage system capacitors degradation, heating up of beamline components, other ambient temperature effects, etc.

These drifts are often substantial enough that operators (or automated algorithm) need to re-tune the accelerator. Further, the sources of drift may not have a direct measurement available or may be challenging to determine out of the many signals. For example, at Stanford Positron Electron Asymmetric Ring accelerator, the orbit settings needed to often be re-tuned due to drift; an investigation of sources of drift found the closest correlation with an ambient temperature sensor, indicating bulk thermal behavior and expansion and contraction of the injector and ring may be responsible for the orbit drift [87].

Figure 2 shows drifts due to both known beam setting changes and gradual drifts caused by unknown factors in the measured beam current data at SNS accelerator. The figure summarizes a time series of beam current data for each pulse into its mean for an easier visualization. Similarly, figure 3 shows impact of gradually drifting data due to slow variation in upstream inputs (laser distribution, RF cavity phase and amplitude) and potentially other unknown changes on ML model predictive performance. It clearly demonstrate that the ML model fails when the data distribution drifts away from the training data distribution.

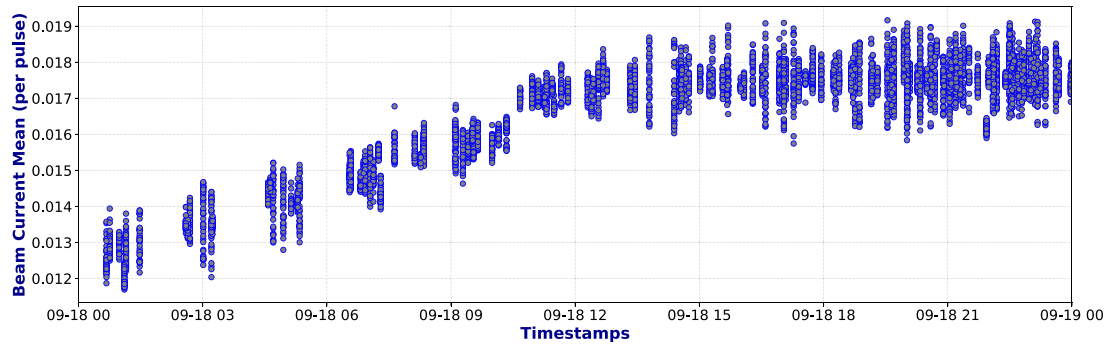


Figure 2. Drift observed in beam current data obtained at SNS accelerator. The y-axis represent mean of the beam current waveform per macro-pulse. Though, it is a harsh summarization of full macropulse about 100 K measurements long, it clearly demonstrates the drift in the measurements.

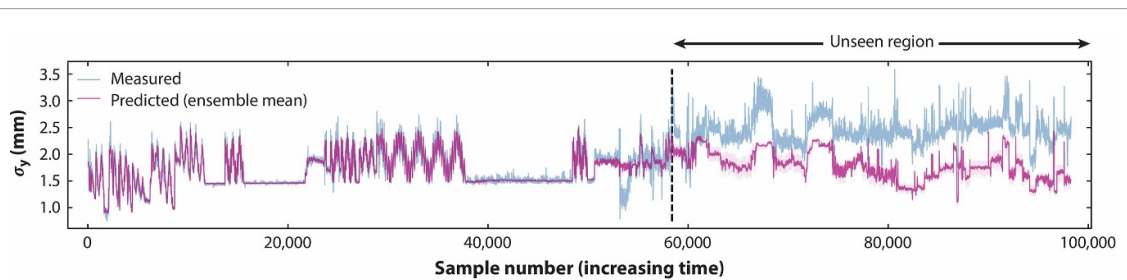


Figure 3. Impact on ML model prediction performance of drift due to slow variation in upstream inputs (laser distribution, rf cavity phase and amplitude) and potentially other unknown changes from a configuration change at a small accelerator. All data shown is test data, but during the period before the marker, there is corresponding training data. This shows the impact of drift on ML model predictions (in this case of the beam size). Reproduced from [86]. CC BY 4.0.

3.2. Low latency requirements, large volume of data, and large number of variables

Particle accelerators are often running at high repetition rates, for example a pulsed accelerator such as SNS runs at 60 Hz, and CEBAF provide a continuous beam of electrons making one round of $\frac{7}{8}$ mile long path in less than a microsecond. Deployable ML models should provide timely responses in order to be actionable [31]. ML models deployed for decision making require precise synchronization with accelerator operations.

On the other hand, accelerators produce huge amounts of data collected from many diagnostics sensors such as BPM [88], and DCM [89]. Anomaly prediction and just-in-time control models require swift processing of the data. Collection, tagging, synchronization across sensors and storage of large volume of data requires a significant amount of resources including software infrastructure and storage server. Achieving low-latency processing can be difficult with the computational demands of large deep learning models.

Particle accelerators are paradoxically limited in number of detailed diagnostic sensors due to many reasons including cost, physical space, etc. However, accelerators also in some cases store hundreds to thousands of measurements at each moment in time, spanning devices across the accelerator. The high dimensionality of these systems can make it challenging to build models that account for the numerous sources of potential changes over time. Data cleaning to prepare for model training and inference can also be challenging due to the dimensionality and the number of different ways signals on the accelerator can fail. Many diagnostics devices operate in noisy environments, leading to significant signal interference. These factors complicate data curation and the training of ML models. A key challenge is balancing the use of more available signals to improve model performance against the risk of introducing failure points if those signals become unreliable. Additionally, data distribution drift over time can make it difficult to maintain a consistent model unless the problem is explicitly treating as time-dependent. These factors play a critical role in selection of appropriate continual learning method as described further in this paper.

3.3. Limited infrastructure to support ML deployment and maintenance

While AI/ML research is often well-funded, the software and infrastructure development required to robustly deploy AI/ML in operations is often neglected. For some applications, the accelerator computing, hardware, and networking infrastructure must be enhanced to enable the data tagging, storage, and movement

capabilities to support AI/ML. For routine deployment into operations, a workflow to update AI/ML models must be implemented in such a way that it can be used by physicists and operators and not just by experts. Integration of compute-intensive AI/ML approaches into operation can require linking the accelerator control system to HPC systems.

Existing particle accelerators and their infrastructure were not designed to accommodate ML deployment. New infrastructure will be required to synchronize data from multiple sensors for any global ML model deployment. Furthermore, many particle accelerators rely on established software systems that may not be designed to accommodate modern ML frameworks. Integrating ML models into these legacy systems can require extensive modifications, which can be challenging and time-consuming.

4. Continual learning

Continual learning is a dynamic process of continuously updating and refining models to adapt to changing conditions, thereby mitigating the impact of concept drift [90]. Continual learning is an active area of research within ML, presenting a paradigm shift from traditional static learning approaches. In contrast to traditional ML paradigms, where models are trained on static datasets and assumed to function within stable environments, continual learning requires that models possess the capability to seamlessly adapt and acquire knowledge from a continuous data stream over an extended time-frame, all while preserving insights gained from previous experiences. This ability replicates the mechanisms of human learning, allowing machines to progressively accumulate knowledge, adjust to evolving tasks, and maintain performance within dynamic environments. Here, task refers to a distinct learnable concept with its own data distribution and objectives within a continual learning setting. As such, *task boundaries* refers to the point where observed data shift from one task to another where applicable. In practical scenarios, often task boundaries are blurred. This iterative approach can be linked to a maintenance regimen for ML models in production environments, where the models are continuously updated to maintain optimal performance. As ML models are updated with new data from a different distribution than the previous training data, it introduces a stability vs. plasticity dilemma [91]. Here, stability refers to the ability of the model to retain its performance on previous data distributions, while plasticity refers to the flexibility of the model in learning new data or concept. An ideal algorithm would maximize both, however achieving a good balance between stability and plasticity is challenging. When given new information, a model with high plasticity may forget previously acquired knowledge. This phenomenon is referred to as catastrophic forgetting [41]. On the other hand, if a model has high stability, it may be too rigid and may not be able to acquire new knowledge.

4.1. Data drift types and detection

Data drifts can be categorized based on their periodicity, speed, and scope. **Periodic** drifts exhibit regular recurrence, while **irregular** drifts lack any discernible pattern. Based on speed, drifts can be classified as sudden, gradual, or incremental. **Sudden** drifts occur rapidly, leading to a new data distribution. **Gradual** drifts involve a transitional phase where the original and new data distributions coexist. **Incremental** drifts occur slowly and subtly, making them challenging to detect. Based on scope, drifts can be classified as local or global. **Local** drifts impact specific subsets of the data, while **global** drifts affect the entire dataset. Local drifts often indicate changes in the data collection or shifts in the underlying sub-processes producing data for the local shifted region. Whereas global drifts implies a fundamental shift in the overall characteristics of the data, impacting all regions or subsets equally. **Recurrent** drifts follow repeating patterns, often driven by seasonal variations or cyclical trends. These types of drifts are often correlated with Periodic drifts. **Evolving** drifts are continuous and non-periodic, reflecting gradual changes in the underlying data-generating processes. Evolving drift can result from various factors such as machine degradation in particle accelerators, or gradual environmental changes.

Detecting drift in ML is essential for maintaining the performance and reliability of models deployed in dynamic environments where the underlying data distribution may change over time. Several methods, including window-based [92], predictive error-based [93], and OOD uncertainty-based [94] drift detection methods, are leveraged to detect drifts in data. Window-based methods maintain a sliding window over the most recently acquired data and utilize statistical tests and/or distance-based metrics to detect deflections in data distribution. Predictive error-based methods monitor the predictive performance of the deployed model and detect model degradation. It is important to note that this methods can only be used when model performance metrics can be calculated in real-time and do not rely on a delayed labeling system. This method can also detect other potential issues with the data pipelines that may cause model degradation. Another method called OOD uncertainty-based drift detection leverages the robust uncertainty quantification capabilities of ML models such as GP. The underlying hypothesis is that OOD uncertainty

would be high when OOD samples are fed to the ML model. Detailed description of these methods are provided in appendix A.

4.2. Types of continual learning approaches

Continual learning encompasses a diverse set of methods designed to enable ML models to adapt to new data over time while retaining prior knowledge. Each approach offers unique strengths and trade-offs, often influenced by constraints such as latency requirements, memory availability, and the nature of data drifts encountered in a given application. Memory-based methods [95] retain past experiences either as raw data or generated samples. Techniques such as experience replay [96] including prioritized variants that assign priorities to data samples based on a predefined criteria [97], generative replay [98], and feature replay [99] are useful for preserving previously learned information but can be resource-intensive. Architecture-based methods achieve continual learning by structurally modifying the network. Techniques include model decomposition [100], modular networks [101], and parameter isolation [102], each assigning different parts of the model to different tasks, thereby minimizing forgetting. Regularization-based methods impose constraints during training to prevent catastrophic forgetting. This includes parameter regularization techniques like EWC [41], MASs [103] and function regularization methods such as LwF [104] that leverage knowledge distillation [105] techniques. Gradient projection methods modify the gradient updates to avoid interference between new and old tasks. Techniques like GEM [106] and OGD [107] are effective in preserving prior knowledge while learning new data. Ensemble-based methods employ multiple models to tackle new tasks while preserving old knowledge. These include both online [108] and block ensemble [109] strategies, balancing flexibility and scalability. Meta learning focuses on enabling models to quickly adapt to new tasks using prior experience, typically in few-shot settings. Both offline [110] and online [111] meta learning variants exist and are particularly useful for low-data regimes. Online continual learning [112] addresses real-time scenarios where data arrives in a stream and can only be processed once. These methods are well-suited for dynamic environments like particle accelerators where low latency and rapid adaptation are critical.

A detailed description of each of these methods, including their mechanisms, advantages, limitations, and practical examples, is provided in appendix B.

5. Deployable ML for accelerators and supporting infrastructure

Traditional deployment of ML typically involves a static model that is trained on a fixed dataset before being deployed in a production environment. These deployment strategies typically perform periodic retraining of the ML models on new data that can lead to catastrophic forgetting. Furthermore, the response to changes in data distribution can be slow, potentially degrading model performance until the next scheduled retraining cycle. In contrast, continual learning facilitates a dynamic learning process where models continuously adapt to new data and operational conditions. This method allows for incremental updates without retraining from scratch, employing strategies such as fine-tuning and memory consolidation to retain prior knowledge. Continuous monitoring of model performance enables rapid adaptation to drifts in data distribution, enhancing the resilience and relevance of ML applications. The continual learning framework thus offers a more robust and scalable solution for environments characterized by high variability and large data volumes, making it particularly suited for applications within particle accelerators.

5.1. Continual learning workflow

Traditional deployment setups typically include a four step loop between data collection, integrity checks, model inference and decision making. Adapting ML models via continual updates requires parallel pipelines for both offline and online continual learning as shown in figures 4 and 5 respectively. Offline continual learning is more suited for use cases with slow and occasional drifts. As depicted in figure 4, it includes data logging and continuous monitoring to trigger model adaptation when model degradation is detected, fine-tuning of the model, testing the new instance of the model in both offline and online (in shadow) mode before potentially replacing previous model.

In contrast, as shown in figure 5, online continual learning involves continuous model update and do not rely on drift detection triggers. However, due to online nature, if a supervised ML technique is used training labels are needed immediately (typically from an automated expert system) for model update. This is more suitable for use-cases with continuous drifts and where either unsupervised learning based models are used or training labels are available immediately.

Particle accelerators produce a huge amount of data, due to the amount of information that is accumulated over several years, across multiple runs. As such, many accelerator facilities only store a small amount of relevant data for future analysis, or save data at a lower rate due to resource constraints. For

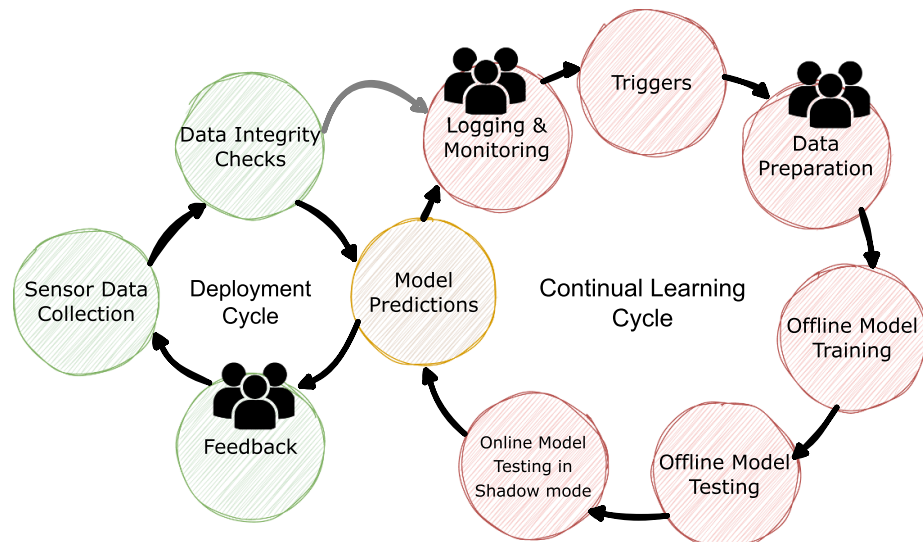


Figure 4. Offline continual learning pipeline with potential human in the loop in parallel to a typical deployment pipeline.

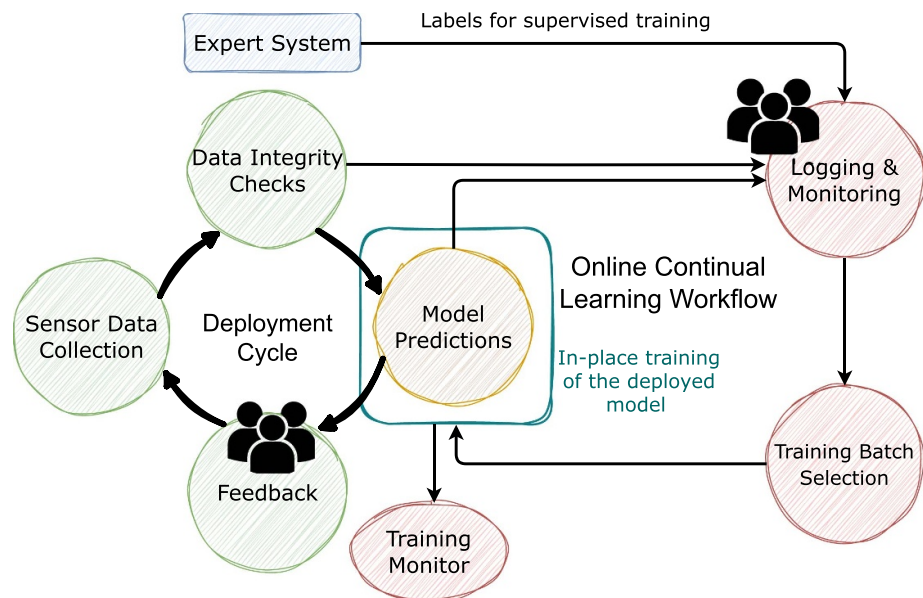


Figure 5. Online continual learning pipeline with potential human in the loop in parallel to a typical deployment pipeline.

facilities that do save comprehensive accelerator operation data at regular intervals, sorting through the data can become difficult. With recent surge in application of ML solutions in this space, many researchers have developed custom solutions to store desired data for training and evaluation of models. For continual learning and model adaptation, depending on the selected approach, data logging becomes a necessity. However, it can be limited to certain time-frame and data samples can be filtered to avoid storing uninteresting samples. For example, in case of anomaly prediction models where normal samples are very similar in distribution, ML model adaptation may only need a small subset of normal data samples.

While the model is providing predictions, the monitoring module continuously observes all the input samples and model predictions to detect any drift in the data and model degradation. The selection of drift detection methods depends on underlying application. For example, for cases where labels become available immediately after data collection, predictive performance based metrics such as model error [93], and average accuracy in case of classification models [113] can be very effective. However, in cases where decision making process influence the availability of labels, it becomes challenging. For example, in anomaly prediction applications in particle accelerators where beam is aborted when the model predicts an upcoming anomaly, it is challenging to know if the alarm was false because the beam has just been aborted. In such

cases, some proxy metrics are required to monitor predictive performance. Window-based correlation methods that do not consider model predictions but monitor input data distributions to detect drifts can also be used. When a drift is detected, model adaptation needs to be triggered. This step simply invokes a parallel process to start a training cycle. This training cycle may train a new model or fine-tune the model using existing weights and biases.

In offline continual learning, when model adaptation is triggered, it launches a parallel process responsible for data preparation, model re-training and testing. The data preparation is entirely dependent on the model adaptation approach. For example, memory-based methods requires all the data including previous data distributions which would require storage of previously collected data. This can be improved up-to some extent by using a clever sampling technique to avoid storing all the data. On the other side, regularization based adaptive methods does not require data from previous data distributions. The training dataset can be divided into three sets, namely training set, validation set, and test set. The test set in this case can be used to evaluate the ML model offline once trained/fine-tuned. It is important to note that in many applications where we desire the model to retain previous knowledge, the model should be tested on data sets from previous distributions to ensure it maintain predictive performance. If a new instance of the model provides better performance than previous version on offline datasets (including both previous distributions and new distribution), it should be tested online in shadow mode to make sure it provides better predictions on incoming data before replacing the old model.

In the context of online continual learning, model adaptation is a continuous process that typically includes newly available data along with a subset of relevant previous data in each batch. This approach helps the model adapt to drifting data and avoid catastrophic forgetting. As the deployed model is continuously updated in-place, this workflow typically requires a training monitoring dashboard to ensure that correct batches are fed into the model training process and to observe loss curves and performance metrics.

5.2. Supporting infrastructure

The successful implementation of continual learning in particle accelerators requires a comprehensive infrastructure that seamlessly integrates hardware, software, and organizational components. The infrastructure can be divided into two parts, namely hardware and software.

Hardware requirements can be further modularized into two sets, one set for running online inference and feedback similar to traditional deployment cycle and second set of hardware to host continual learning processes. The hardware for executing rapid inferences in real-time applications is dependent on the computational and latency needs for the application. If very low latency is needed, the hardware for inference may need to be deployed on the 'edge,' i.e. close to the accelerator devices themselves. This could include application specific integrated circuits/field programmable gate arrays if latency requirements are extremely low (e.g. less than ms). For applications with more lenient timing restrictions but only modest compute requirements, CPU/GPU systems directly on the accelerator controls network may suffice. In the context of continual learning, to ensure that data can be effectively utilized for future model updates, a dedicated storage server is typically needed for logging and archiving operational data. Existing control systems may have a standard archiver, which may save data locally or on a separate network. Where the data is stored and how much historical and immediate data is used in the continual learning application will have an impact on where the actual continual learning process occurs (e.g. locally or on a separate network).

For applications with lightweight ML model and modest computational requirements, a local CPU/GPU machine can be hosted within accelerator network to facilitate continual learning cycle including monitoring, model re-training, and testing. In contrast, for applications requiring more compute or access to larger data sets, interfacing with HPC systems such as local compute clusters may be needed. This does come with potential latency increases and considerations for how job scheduling is handled particularly in the context of large clusters with shared users. For use in online and continual learning applications, some HPC systems may have a reservation system or prioritized queue. tools like Kubernetes [114] can help support long-running jobs on clusters, for applications like streaming data processing and continual learning. Both quasi-real-time inference (e.g. on the order of seconds) and interaction with the control system, as well as continual learning loops can be conducted on such HPC systems, provided a sufficient network bandwidth and speed is available. For example, SLAC has connected its control system to its local HPC facility, the SLAC Shared Science Data Facility (S3DF), where compute intensive particle-in-cell simulations of the accelerator can be run from machine readings in quasi-real-time, suitable for use in a control room (e.g. minutes). Using the same hardware and software infrastructure, ML models are able to be deployed and put into a continual learning loop. Similarly, SLAC uses the NERSC SuperFacility API [115] to enable photon science users to quickly analyze data and display results in the control room; this is now being expanded to include online retraining of ML models used in analysis. SLAC and LBNL are working on connecting online modeling / DT

Continual Learning Methods		Constraints				
Methods	Sub-category	No/Minimal Forgetting on Previous Tasks	Previous Tasks Datasets Required	Small Dataset for New Tasks	Task Boundaries Required	Low Latency
Memory-based Methods	Experience Replay	✓	✓	✓		
	Generative Replay	✓	Replaced by Generative Model(s)	Generative Model Training Require Enough Data Samples	✓	
	Feature Replay	✓	Replaced by Extracted Features	Feature Extractor Training Require Enough Data Samples	✓	
Architecture-based Methods	Model Decomposition	✓			✓	
	Modular Networks	✓			✓	
	Parameter Isolation	✓			✓	
Regularization-based Methods	Parameter Regularization			✓		✓
	Function Regularization			✓		✓
Gradient Projection Methods		✓	✓	✓	✓	
Ensemble-based Methods		✓			✓	
Meta Continual Learning (Few Shot)				✓		✓
Online Continual Learning				✓		✓

Figure 6. A guide to selection of appropriate continual learning method(s) for a given applications based on given constraints.

infrastructure to NERSC through the SuperFacility API. The US Department of Energy is planning to further enable these types of connections through its Integrated Research Infrastructure.

To support continual learning, software infrastructure is also needed. The sophistication of this may vary depending on whether edge compute, compute on the controls network, or separate HPC clusters are used. Tools such as MLFlow [116] help with tracking and updating of ML models, and come with an associated model database. Tools such as Prefect [117] and AirFlow [118] can aid workflow orchestration, for example to keep track of and spawn processing associated with the ML monitoring and updating workflow automatically. Finally, tools like Kubernetes can help with orchestration of containerized applications, which is especially useful on shared HPC clusters. Some HPC facilities, such as NERSC, have custom APIs including for quasi-real time interaction.

Network requirements are critical for continual learning infrastructure. High-speed communication links are essential for facilitating data transfer between hardware components, as well as for the timely acquisition of sensor data. A low-latency network is vital for minimizing delays in data processing and model inference, ensuring that real-time applications can operate smoothly. This requirement maybe trivial to deal with for continual learning on edge hardware or CPU and/or GPU machine inside accelerator network and close to the sensors. However, it is more complicated for continual learning on HPC.

5.3. Selection of continual learning method(s)

ML model deployment use cases can broadly be classified into two categories, i.e. online deployment that provide ‘real-time’ decision support and offline as a surrogate. For online deployed models, one of the main deciding factors for selection of continual learning method is latency. When drift causes the model to become obsolete, and the continual learning process is updating it to replace the old one, there is a period during which inferences can not be used for decision making. This is due to the fact that the predictions from the old model may no longer be reliable and should not be used for decision making. This interruption can be referred to as ML downtime. It is desired to minimize the ML downtime by enabling quick adaptation of the models in real-time use cases. In contrast, for offline models, this requirement can be relaxed. In this section, we revisit the applications of ML in particle accelerators as discussed in section 2 and provide insights on which continual learning method(s) are most appropriate to handle the concept drifts. This is mainly guided by the constraints on a given application and the properties of various continual learning methods as described in figure 6.

5.3.1. Anomaly detection

Anomaly detection constitutes a significant domain of application within particle accelerators. This includes detection of gradually failing equipment as well as rapid deflection in beams. ML models developed for

anomaly detection or prediction are run online in real-time. Such models that provide timely predictions are essential for real-time decision making that are crucial for accelerator operation. For example, outputs from an anomaly predictions model are used to abort the beam to avoid damage to the equipment by the deflected beam. As such, rapid adaptation to distribution drift is critical to avoid ML downtime.

In anomaly prediction applications, as the accelerators are running continuously, there are multiple samples collected every second (depending on nature of machine and their frequency). For example, SNS at ORNL produces 60 samples of beam current time series waveform every seconds where each waveform contains about 100 K measurements [31] that is equivalent to 6 M points in one second. These data samples can be continuously collected and stored but require a large storage and significant computing power to process. Machines that meet these computational demands are typically not integrated within the accelerator network. As such, coupled with low latency requirements, model adaptation approaches that require data from previous tasks such as memory-based methods are less suitable in such cases. It is also desirable to adapt the models with limited number of samples for new task, especially for supervised learning ML methods that require labels. If a continual learning method requires a large number of samples for new task, it artificially increases the ML downtime until a sufficient number of samples has been collected and the model can reach adequate performance. This is because predictions from the old model may no longer be useful for decision making. Another constraint that is critical when choosing a continual learning method is whether clear task boundaries are defined. As discussed above, particle accelerator data can have two types of distribution drifts, one caused by known changes and others due to unknown/non-measured factors. Clear task boundaries can be defined in cases when distribution drift is due to known changes in accelerator settings, however it is challenging in the later types of drifts. As such, methods that may require clear task boundaries may also be less suitable in this use case.

This leaves us with three options, regularization-based methods, meta learning, and incremental learning. Each of these three methods suffers with some level of forgetting on previous tasks. With unknown drifts in the data, it is generally acceptable to tolerate some level of forgetting on the previous data distributions. However, instead of using one of these methods in its pure form, it is advisable to fuse them for this use-case. For example, meta-learning can be used to group the tasks based on known drifts. To handle unknown drifts within these groups, incremental regularization methods can be applied. This new hybrid approach could potentially provide several benefits, the grouping leveraging meta learning will divide many beam settings into several groups based on their similarity to avoid small number of samples in base-learners. Single base-learners per group can be quickly adapted with regularization based methods in conjunction with incremental learning to reduce ML downtime.

5.3.2. Optimization and control

Optimization applications in particle accelerators range from optimization of the individual sub-systems to optimization of the global settings for various equipment to achieve a desired stable beam delivery. In addition, particle accelerators can greatly benefit from autonomous control methods. In fact, many facilities are aiming towards self-driving particle accelerators. For autonomous ML operational cycle, continual learning is of great importance to maintain ML performance with changing conditions.

As discussed in section 2, many ML models have been used for optimization and control including BO, RL, and MPC. Some of these approaches, such as BO, learn a new model from scratch at each run, whereas others, such as RL, and MPC may use a persistent ML model for the control policy and/or the value function. All the three algorithms, BO, RL, and MPC can use a learned system model to improve performance, and this model needs to be adapted over time as the machine changes. In addition, RL, and MPC can also benefit from continual adaptation of the policy (controller) model with changing conditions.

When particle accelerators are restarted for a new run, ML models may need to be adapted from previous runs due to drifts in the data or setting up a new set of conditions (e.g. new beam energy). In high power machines the models are run offline, or during a low power operation before the full power operation of the accelerator. For systems that operate only in a few target beam configurations, some level of forgetting on previous data distribution is generally acceptable while adapting the models. However, for machines that are put into many different operating conditions, it is desirable to retain some learning from previous operating conditions while also adapting to new ones. In addition, for online optimization, to minimize ML downtime, it is desirable to choose continual learning method that has low latency and does not require large amount of data samples for the new task. Moreover, task boundaries can be defined for distribution drifts due to known changes but not for the drifts due to unknown factors.

With these constraints, memory-based methods are suitable for optimization applications with a clever sampling techniques such as priority replay [97] or Hindsight experience replay [119] to achieve relatively low latency. It is important to note that traditional circular buffer used in RL will not be sufficient as the data samples from all the previous distributions should be retained in the buffer to limit forgetting. This is further

complicated by the slow and gradual drifts from unknown sources where distribution boundaries can not be defined. As such, custom solutions are needed to effectively maintain samples to cover the entire area of the input space. In addition, gradient projection can be a good choice as it can allow quick adaptation when small dataset is available for new task.

For online control applications, meta-learning can be used to continuously adapt the agent/controller with drifting data. Meta-RL [120] has already been explored in other fields, though for offline multi-task learning, it can be adapted for continual learning with minimal modifications. In addition, for methods such as DRL, and DMPC that involve deep learning models, regularization based methods can be combined with online continual learning methods for continuous adaptation of the models as this is naturally fits into the RL/MPC framework.

5.3.3. Surrogate models and data driven DTs

Surrogate models are typically trained offline on real data. These models can be used for several applications including building of environments for offline training of optimization and control algorithms such as RL or MPC. These models are often trained on historical data, however, new data distributions may need to be introduced from outside of the training data distribution. For example, a surrogate model trained on historical data may need to be updated periodically with new measured data to keep up with real drifts in the physical machine. It is more important in data driven DT models that need to keep up with the physical system and continuously adapt to drifting system behavior. Such model adaptations also occur offline and are more flexible than online model adaptation.

In these cases, no forgetting on previous tasks is desired to be able to reproduce previous results up-to a certain time horizon in the past and to be able to learn new data distributions caused by time-varying drifts. Dataset from previous data distributions are typically stored and are available. As such, to maximize performance and limit forgetting, a memory-based approach should be preferred. However, if the data volume grows and cause significant challenges in using all the data for fine-tuning, a priority based replay can be used [97] to both limit the amount of data stored and make re-training faster. In addition, as more and more drifted data distributions are added, the model may get saturated. To address this, architecture based continual learning methods (such as progressive neural networks) can be combined with replay to add more capacity to the model.

In addition, these models can greatly benefit from the estimation of OOD uncertainty to inform the user when test data samples arrive from outside of the training data distribution. The uncertainty estimation can also be used as a drift detection mechanism.

5.3.4. Virtual diagnostics

Virtual diagnostic models are often run in real-time in sync with the accelerator operation to provide indirect measurements relevant to the beam. If the predictions from virtual diagnostics model are used for real-time decision making, low latency is typically required for the model adaptation to reduce the ML downtime. Whereas, if the virtual diagnostics model is used for offline analysis of the acquired data, some latency in model adaptation is usually acceptable.

Within a shorter temporal scope, where data drift due to non-measured factors remains minimal, it is feasible to define data distribution boundaries. This is especially critical for machines that transition between various settings in a shorter time-span. In such setups, retaining information on prior data distributions within a certain temporal horizon is advantageous, as the machine frequently reverts to previous settings. Conversely, over longer durations where data drifts due to unmeasured factors are evident, establishing distribution boundaries becomes challenging. In addition, some level of forgetting is acceptable on data distributions that are relatively older as these may have been compromised by unknown drifts.

Similar to real-time anomaly prediction, in this use case, hybrid continual learning approaches are most suitable. A time-based priority data buffer can be maintained per beam setting instance to store the data within a fixed time horizon. This buffer can be utilized in memory-based continual learning methods to enable faster model adaptation with minimal forgetting on data distributions that are recent in time. The priority can be assigned to samples in the buffer based on given figure-of-merit and existing model's predictive error. In this approach, there is a trade-off between number of beam settings that can be included (and the size of buffers) and adaptation speed. As the number of beam settings may grow, the model adaptation may slow down. In such cases, Ensemble-based methods can be combined to incrementally learn a model per beam setting that is fine-tuned from a base model that is aware of all the beam settings. This approach is advantageous in learning new unseen beam settings faster with minimal number of samples as the model training leverages base model instead of training from scratch. Another option is to use architecture based methods in place of ensemble based method to allow better scalability in setups with limited computing and memory resources. In addition, models and data distribution that are relatively older

and are known to have unknown drifts can be discarded as they may no longer be valid even if the accelerator transition to those exact settings.

6. Conclusion and outlook

Applications of ML models in particle accelerators struggles significantly with data distribution drifts. As such, there is a huge gap between the number of ML solutions being developed and number of these solutions being used long-term in operation. This domain can significantly benefit from continual learning methods.

In this paper, we have provided a guide on development of adaptive ML models tailored for various applications within particle accelerators. We delved into the existing landscape, offering a brief overview of the existing ML applications. Moreover, we explored the challenges associated with deployment of these models in operational environment, specifically emphasizing the relevance to continual learning. Subsequently, we describe a detailed analysis of existing continual learning methodologies, along with their limitations and constraints pertaining to applications in fast-paced accelerator space. We have provided a mapping between various application use-cases in accelerators and continual learning methods for the seamless automation of model adaptation in response to the time-varying drifts in the measurements whilst introducing novel hybrid approaches that hold the promise of optimal suitability and groundbreaking potential. In addition, we have provided a guidance on the robust infrastructure requirements based on our experience from SLAC national laboratory and SNS accelerator. With this paper, we have attempted to open up a new direction to manage data drifts and inspire more research efforts towards deployable continual learning for particle accelerators.

Data availability statement

The data cannot be made publicly available upon publication due to legal restrictions preventing unrestricted public distribution. The data that support the findings of this study are available upon reasonable request from the authors.

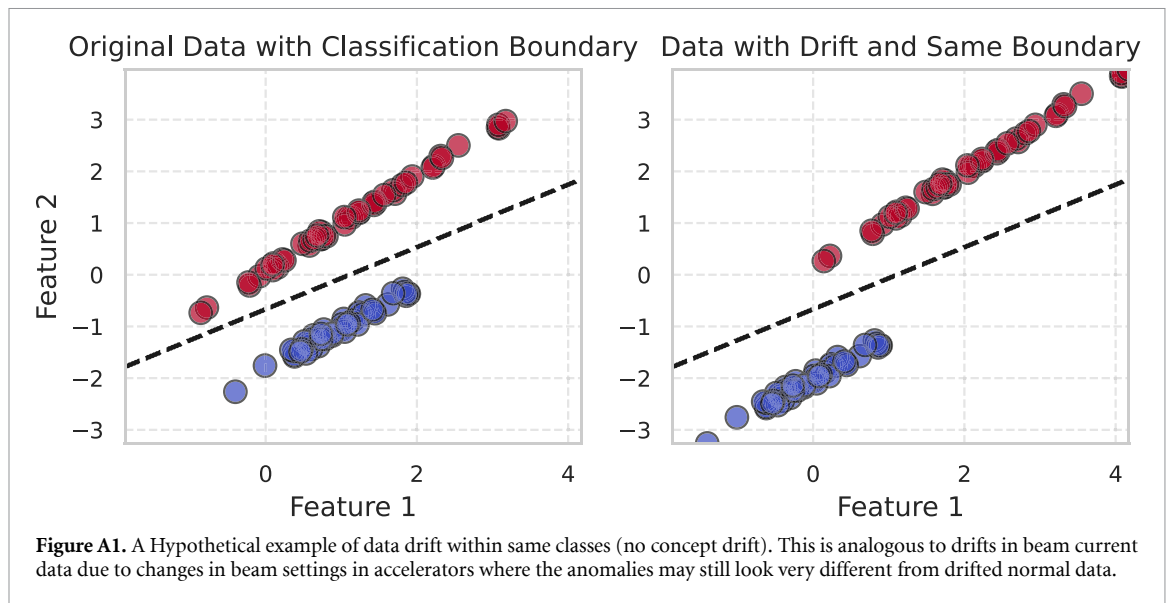
Acknowledgment

This work was partially supported by the DOE Office of Science, United States under Grant No. DE-SC0009915 (Office of Basic Energy Sciences, Scientific User Facilities program). This manuscript has been authored by Jefferson Science Associates (JSA) operating the Thomas Jefferson National Accelerator Facility for the U.S. Department of Energy under Contract No. DE-AC05-06OR23177. Oak Ridge National Laboratory is operated by UT-Battelle, LLC, under Contract DE-AC05-00OR22725. The contributions from Auralee Edelen are supported by SLAC National Accelerator Laboratory, under Contract DE-AC02-76SF00515 for the US Department of Energy. The US government retains, and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>)

Appendix A. Drift detection methods

A.1. Window-based methods

A number of window based drift detection methods are explored by the researchers. Simple methods include sliding window statistical tests such as Kolmogorov–Smirnov test [121], chi-square test [122], Wasserstein distance [123] that can be used to detect any deflection from normal trend with a threshold. In a slightly different approach sliding window statistical distance (Jensen–Shannon divergence [124], Kullback–Leibler divergence [125], Euclidean distance, earth movers distance etc) can be compared with a fixed reference window. ADWIN [92] is another drift detection method used in data streams to monitor changes in the underlying statistical properties of the data. It is particularly useful for detecting concept drift, which refers to changes in the relationship between input features and target variables over time. The basic idea behind ADWIN is to maintain a sliding window of fixed size over the incoming data stream and monitor statistics within that window. As new data points arrive, ADWIN continuously updates its assessment of whether the data distribution within the window has remained stable or if a significant change, indicative of drift, has occurred. ADWIN is known for its ability to detect drift with low computational overhead and adapt to varying rates of change in the data stream. It has been widely used in applications such as online learning,



real-time monitoring, and adaptive systems where timely detection of concept drift is critical for maintaining model performance.

A.2. Predictive error-based methods

It is important to note that the data drift do not always implies concept drift or model degradation. Figure A1 shows one such example where data drifts within the individual classes. In this example, data drifts, however no concept drift is observed as the previous classifier is still valid. Drift detection methods that only focuses on data distribution may trigger unnecessary model re-training in such cases. This is the reason why predictive error based drift detection methods has become more popular. Predictive error based methods can also detect other potential issues with the data/inference pipelines that may cause performance degradation. As such, predictive error based drift detection methods are proposed. One such method is called drift detection method [93] which is later extended to early drift detection method [126] and McDiarmid drift detection method [127] to improve the performance on gradually drifting data. In addition, window based methods discussed above can also be used to monitor the predictive error-rate with a sliding window and trigger when the rate crosses a pre-defined threshold.

A.3. OOD uncertainty based drift detection

OOD uncertainty-based data drift detection method operates by assessing the uncertainty associated with model predictions on incoming data samples. The core idea is to analyze whether the incoming data points belong to the same distribution as the training data, using uncertainty metrics derived from the model's predictions. ML models has shown to provide OOD uncertainties with Bayesian neural networks [128], deep GP approximation [129], and GPs [130]. In this approach, the uncertainty scores produced by the model are continuously evaluated for the predictions on new data points. If the uncertainty exceeds a predefined threshold, it indicates that the data point may be OOD, suggesting a potential drift in data distribution. This threshold can be calibrated based on historical performance metrics or statistical analysis of uncertainty scores [94]. OOD based uncertainties are not yet fully leveraged in particle accelerator applications as many ML solutions are developed without inclusion of uncertainty quantification on predictions. However, OOD uncertainty has the potential to be very useful in validating predictions of the models, especially in real-time decision making processes as outlined in ML based calibration and control of GlueX detector at Jefferson Lab [131].

Appendix B. Types of continual learning methods

B.1. Memory-based approaches

Memory-based continual learning methods have shown promising performance in addressing the challenges of catastrophic forgetting, where a model trained on new data may lose the knowledge acquired from previous tasks. These methods incorporate past experiences by replaying or revisiting old data during the training process, thus reinforcing previously learned representations while learning new information.

One common approach is **experience replay**, which involves storing a subset of past data and integrating it into the training of new tasks [96]. This technique has been effectively utilized in RL, where agents retain and reuse past experiences to improve performance [132]. In the context of supervised learning, replay can be implemented by maintaining a buffer of data points from previous tasks and sampling them alongside new task data during training. Priority experience replay [97] enhances learning efficiency by replaying experiences based on their importance, rather than sampling them uniformly. Samples that indicate greater learning potential, are prioritized, ensuring the model focuses on more informative samples. Another approach that focuses on exploiting negative experiences to extract useful knowledge is hindsight experience replay [119], which has been explored in practical application of continual learning for sequential object manipulation [133].

Due to high memory requirements of experience replay methods, **generative replay** approach has been developed. These methods typically employ generative models, such as VAE or GAN, to approximate the data distribution of past tasks and replay samples from this distribution as if they were real [98]. By doing so, the model can consolidate knowledge from previous tasks while learning new ones, enabling it to perform well across all tasks without requiring access to original data. Generative replay is particularly beneficial in privacy-sensitive or resource-constrained scenarios, where storing real data from earlier tasks may not be feasible [134].

Another replay approach is **feature replay** that addresses catastrophic forgetting by replaying learned feature representations instead of raw data, making them computationally efficient and less prone to privacy concerns. In this approach, the model stores or generates intermediate feature representations of data from previous tasks, which are replayed during training on new tasks to preserve knowledge while incorporating new information. Unlike generative replay, feature replay avoids the complexity of generating raw data and focuses on maintaining compact and semantically meaningful representations [99]. This method is particularly effective in architectures with encoder–decoder structures or pre-trained feature extractors, enabling scalability and efficiency in sequential task learning. It is important to note that if the pre-trained feature extractor model is affected by data drifts, the quality of the extracted features may degrade causing error propagation in the continual learning process and can lead to forgetting of previous knowledge. This method is typically used in computer vision where pre-trained embedding models are known to exist for a variety of images.

Recent advancements also include **multi-task learning with replay**, where a model is trained on multiple tasks simultaneously while periodically revisiting past tasks. This approach helps in maintaining performance across all tasks by ensuring that the model does not drift significantly from the knowledge gained previously [41]. The integration of memory-efficient techniques, such as selective replay, further enhances the efficiency of replay-based methods by focusing on the most informative samples from past tasks [135].

Overall, memory-based continual learning methods represent a promising direction for developing systems capable of adapting to new information while preserving prior knowledge, making them essential in applications requiring lifelong learning. However, they have a high requirement of memory and/or computing resources and may not be suitable for low-latency tasks.

B.2. Architecture based approaches

Architecture-based methods modify the neural network architecture to support the storage and retrieval of knowledge over time. Architecture-based methods can be classified into three categories, namely model decomposition, modular networks, and parameter isolation. These approaches allocate specific components of the network to different tasks, enabling efficient retention of learned knowledge while incorporating new information.

Model decomposition divides the network into shared and task-specific components. Shared components capture common knowledge across tasks, while task-specific components focus on task-specific features. By decoupling these aspects, the model minimizes interference between tasks. For example, PNN maintain a set of architectures for different tasks, allowing new networks to leverage previously learned features without overwriting them. This method efficiently transfers knowledge by freezing the weights of previous networks while allowing new layers to learn new tasks [100]. PNNs has been demonstrated in robotic systems, where robots learned to manipulate objects in different environments by building on previously learned skills [136].

NMN consist of multiple subnetworks that can be activated based on the task. Each module learns a specific aspect of the data, which helps in isolating the learning process and improving performance on diverse tasks [101]. A study showed that using modular networks for NLP tasks allows models to efficiently learn new languages while retaining proficiency in previously learned languages [137].

Parameter isolation methods protect subsets of the network's parameters from being overwritten when learning new tasks. These approaches often rely on masking, freezing, or dynamically allocating parameters for each task [102].

In general, architecture-based methods has minimal forgetting on previous tasks by nature as they assign different parts of the network to different tasks and freeze previous knowledge. However, these methods require task boundaries to be effective.

B.3. Regularization based approaches

Regularization-based continual learning methods aim to mitigate catastrophic forgetting by imposing constraints on the model's learning process. Regularization-based methods can be broadly divided into two categories, namely parameter regularization and function regularization. Regularization-based methods are advantageous in that they do not require explicit storage of past data, making them memory efficient. They allow the model to maintain performance across multiple tasks while learning new information. However, these approaches can introduce additional computational overhead and may require careful tuning of regularization parameters to achieve optimal performance [90].

Parameter regularization typically adds regularization terms on the gradients to protect the weights associated with previously learned tasks while allowing adaptation to new tasks. EWC [41] is one of the pioneering methods in this category. It uses a quadratic regularization term that penalizes changes to the important parameters identified during training on previous tasks. Specifically, EWC calculates the Fisher information matrix to determine the significance of each parameter, thereby penalizing updates to parameters that are crucial for past tasks. This method enables the model to learn new tasks while preserving knowledge of previously acquired ones. MAS [103] builds on the idea of EWC by introducing a more adaptive approach to regularization. Instead of using a fixed Fisher information matrix, MAS continually updates a measure of importance for each weight based on the current task. This allows the model to dynamically adjust its sensitivity to changes in the parameters as new tasks are introduced, providing a more flexible means of managing catastrophic forgetting.

In contrast, function regularization operates at the functional level, aiming to preserve the model's output behavior (the function it represents) for previous tasks. This is achieved by explicitly maintaining consistency between the model's predictions for old tasks and those made prior to learning a new task. For example, the LwF method [104] uses knowledge distillation [105]. Instead of requiring access to the original training data from previously learned tasks, LwF leverages the predictions of the original (pre-update) model as a proxy for those tasks. When training on a new task, the updated model is optimized to perform well on the new task at the same time it is also encouraged to produce similar outputs for the old tasks as it did before the new training began. This is achieved through a distillation loss, which measures the divergence between the old model's output probabilities and the new model's outputs on the same inputs (typically, the inputs from the new task). The key idea is to treat the old model's outputs as 'soft targets' and enforce that the current model maintains consistency with them. This loss penalizes significant deviations in the output distributions for previously learned tasks, effectively acting as a regularizer that constrains the shared parameters from drifting too far from those that supported past performance.

B.4. Gradient projection

Gradient projection [138] minimizes catastrophic forgetting by constraining the gradients during the optimization/training. The idea is to ensure that the updates for new task do not interfere with the learned parameters for previous task(s). This is achieved by projecting gradients for the new task to a space that is less likely to conflict with gradients critical for previous tasks, this relies on geometric principles to identify such subspaces. GEM, a gradient projection method stores a small subset of previous task data to use them to constraint gradient updates. It has been developed to alleviates forgetting, while allowing beneficial transfer of knowledge to previous tasks [106]. GEM has been applied in RL for robotics to learn new tasks while retaining knowledge on previous tasks [106]. An extension of GEM called A-GEM simplifies the gradient projection step by ensuring that the averaged gradients of stored samples from previous tasks are aligned with the new task gradient. A-GEM has been tested in healthcare for disease classification across datasets and autonomous driving applications [139]. In a slightly different approach, OGD [107] projects the gradients for new task orthogonal to the previous task(s) gradients. To remove the need of storing gradients in the memory in OGD, more advanced techniques such as Gradient Projection Memory [140], and Trust Region Gradient Projection [141] have been developed by extracting and storing important bases of the input subspace.

B.5. Ensemble methods

Ensemble-based continual learning methods integrate multiple models to improve adaptability and performance in dynamic environments. These approaches leverage the diversity of individual learners to mitigate catastrophic forgetting while maintaining overall model accuracy [21]. Advantages of ensemble methods include enhanced robustness, improved generalization across tasks, and the ability to incorporate

new knowledge without extensive retraining [104]. However, there are some challenges associated such as increased computational overhead, model management complexity, and potential difficulty in ensuring cohesive decision-making among diverse models [142]. Ensemble-based continual learning methods are broadly classified into two categories namely online ensembles and block ensembles.

Online ensemble methods incrementally update models without revisiting past data, ensuring adaptability to non-stationary distributions. Each incoming task or data batch may correspond to a lightweight model that is dynamically integrated into an ensemble. Techniques such as online boosting or bagging are common examples, where models are added or re-weighted based on their performance on new data [108]. Online ensembles are particularly effective in scenarios with frequent distribution drifts, but their scalability and memory constraints remain a challenge in long-term learning [143].

In contrast, block ensemble methods operate in discrete phases, where each block corresponds to a specific task or data batch. Instead of incrementally updating a model, separate models or components are trained for each block and then combined into an ensemble. Methods such as expert networks associate individual blocks with specific tasks and use gating mechanisms to select the most relevant expert for predictions [109]. Although block ensembles are robust against forgetting, their computational and memory requirements grow with the number of tasks, limiting scalability.

B.6. Meta learning

Meta learning, often referred to as 'learning to learn', focuses on adapting quickly to new tasks by leveraging prior knowledge gained from previous tasks [144]. This paradigm is particularly useful in scenarios where data for a specific task is scarce, allowing models to generalize better from limited examples. Meta-learning typically involves learning from a distribution of tasks. The idea is to capture the underlying patterns that can be generalized across different tasks. This can be formalized as learning a meta-model that optimizes performance on a variety of tasks, enabling rapid adaptation to new tasks [145]. One common strategy in meta learning is to employ model-based methods, where the architecture of the model itself can adapt based on the learned representations. For instance, MAML is a popular algorithm that trains a model such that it can achieve good performance on new tasks after only a few gradient updates [145]. Another approach involves optimizing the learning algorithms themselves. This can include tuning hyperparameters or even learning optimization algorithms that are more effective for specific classes of tasks [146]. Metric-based approaches utilize learned distance metrics to facilitate quick adaptation. For example, in few-shot learning scenarios, models like prototypical networks learn to compute distances between data points in an embedding space, enabling effective classification of new classes based on a few examples [147].

Meta-learning has shown promise in various applications, including few-shot learning, where models are required to learn from a limited number of labeled examples [148]. This has shown to be particularly beneficial in domains like healthcare and robotics, where obtaining labeled data can be costly and time-consuming [149]. Additionally, meta-learning techniques have been applied to improve the efficiency of RL agents, allowing them to adapt more rapidly to new environments [150]. This includes a recent study on adapting RL to changing conditions in AWAKE project at CERN with the help of meta-RL [151]. The study demonstrate meta-RL's ability in managing partially observable Markov decision processes with evolving hidden parameters prevalent in particle accelerators.

Offline meta learning involves training models on a fixed set of tasks or datasets before deployment. The model learns to generalize across these tasks, and once trained, it can quickly adapt to new tasks during inference without further learning from new data. This approach emphasizes the pre-training phase, where the model accumulates knowledge from a broad range of experiences. Offline meta-learning typically relies on a well-defined distribution of tasks, allowing the model to learn generalizable strategies [110].

Online meta learning refers to methods where the model learns continuously from a stream of tasks or experiences. This approach allows the model to update its knowledge dynamically as new data becomes available, facilitating rapid adaptation to new tasks with minimal retraining. The core idea is to leverage immediate feedback from newly encountered tasks to improve performance on future tasks. Models can adjust their parameters in real-time based on incoming data, making them suitable for environments where tasks are non-stationary [150]. Typically, online meta-learning is employed in scenarios where the model has to learn from a limited number of examples for each new task [152].

B.7. Online continual learning

Unlike offline continual learning where each task has an offline dataset and the model can be updated using the dataset for multiple times, in online continual learning [112] data arrives in streams and can only be trained once. This setup is particularly advantageous in dynamic environments with pure online data and low latency requirements. It seeks to update the ML models quickly and continuously using the current small amount of data, without forgetting the knowledge gained previously. The strategies to address forgetting in

online continual learning follow a similar line as in offline continual learning, which implies that many offline continual learning approaches can be adapted to the online scenario. In the meanwhile, the online scenarios put a special focus on the learning effectiveness of ML models with a few data points.

References

- [1] Wilson E 2001 *An Introduction to Particle Accelerators* (Oxford University Press)
- [2] Peach K, Wilson P and Jones B 2011 *Br. J. Radiol.* **84** S4–10
- [3] Nghiem P A P, Chauvin N, Simeoni W and Uriot D 2012 Beam halo definitions and its consequences (available at: <https://api.semanticscholar.org/CorpusID:56375210>)
- [4] Mobarak M H, Mimona M A, Islam M A, Hossain N, Zohura F T, Imtiaz I and Rimon M I H 2023 *Appl. Surf. Sci. Adv.* **18** 100523
- [5] Larranaga P et al 2006 *Brief. Bioinf.* **7** 86–112
- [6] Rajpurkar P et al 2017 Chexnet: radiologist-level pneumonia detection on chest x-rays with deep learning (arXiv:1711.05225)
- [7] Rangel-Martinez D, Nigam K and Ricardez-Sandoval L A 2021 *Chem. Eng. Res. Des.* **174** 414–41
- [8] Boehnlein A et al 2022 *Rev. Mod. Phys.* **94** 031003
- [9] Grosan C and Abraham A 2011 *Artificial Neural Networks* (Springer) pp 281–323
- [10] Edelen J P and Cook N M 2021 Anomaly detection in particle accelerators using autoencoders *Technical Report* (arXiv:2112.07793 [physics])
- [11] Blokland W et al 2022 *Phys. Rev. Accel. Beams* **25** 122802
- [12] Rajput K, Schram M, Edelen A, Colen J, Kasparian A, Roussel R, Carpenter A, Zhang H and Benesch J 2025 *Mach. Learn.: Sci. Technol.* **6** 025018
- [13] Edelen A, Edelen J, Biedron S, Milton S and van der Slot P 2017 Using neural network control policies for rapid switching between beam parameters in a free electron laser *Deep Learning for Physical Sciences Workshop at the 31st Conf. on Neural Information Processing Systems* (Long Beach, CA, USA)
- [14] Gupta L, Edelen A, Neveu N, Mishra A, Mayes C and Kim Y K 2021 *Mach. Learn.: Sci. Technol.* **2** 045025
- [15] Chaturvedi A R, Hutchinson G K and Nazareth D L 1993 *Decis. Support Syst.* **10** 213–33
- [16] St John J et al 2021 *Phys. Rev. Accel. Beams* **24** 104601
- [17] Scheinker A, Cropp F, Paigua S and Filippetto D 2021 *Sci. Rep.* **11** 19187
- [18] Bayram F, Ahmed B S and Kassler A 2022 *Knowl.-Based Syst.* **245** 108632
- [19] Lu J, Liu A, Dong F, Gu F, Gama J and Zhang G 2018 *IEEE Trans. Knowl. Data Eng.* **31** 2346–63
- [20] Vela D, Sharp A, Zhang R, Nguyen T, Hoang A and Panykh O S 2022 *Sci. Rep.* **12** 11654
- [21] Wang L, Zhang X, Su H and Zhu J 2024 *IEEE Trans. Pattern Anal. Mach. Intell.* **46** 5362–83
- [22] Wang Z, Liu L, Duan Y, Kong Y and Tao D 2022 Continual learning with lifelong vision transformer 2022 *IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 171–81
- [23] Biesialska M, Biesialska K and Costa-jussà M R 2020 Continual lifelong learning in natural language processing: a survey *Proc. 28th Int. Conf. on Computational Linguistics* ed D Scott, N Bel and C Zong (International Committee on Computational Linguistics) pp 6523–41
- [24] Lesort T, Lomonaco V, Stoian A, Maltoni D, Filliat D and Díaz-Rodríguez N 2020 *Inf. Fusion* **58** 52–68
- [25] Rumelhart D E, Hinton G E and Williams R J 1986 *Learning Internal Representations by Error Propagation* (MIT Press) pp 318–62
- [26] Kingma D P and Welling M 2013 Auto-encoding variational bayes (arXiv:1312.6114)
- [27] Humble R, Colochio W, O'Shea F, Ratner D and Darve E 2023 Resilient vae: unsupervised anomaly detection at the slac linac coherent light source (arXiv:2309.02333)
- [28] Gao Y 2023 Initial results of applying an autoencoder to detect anomalies in the air conditioning systems of the brookhaven accelerator complex *Proc. 14th Int. Particle Accelerator Conf. (IPAC'23 -)* (JACoW Publishing) pp 4448–51
- [29] Abadjiev D, Adams T, Adzic P, Ahmad M, Amendola C and Andrews M B The CMS ECAL Collaboration 2024 *Comput. Softw. Big Sci.* **8** 11
- [30] Stern T D, Mishra A K and Keaveney J M 2025 *Computers* **14** 79
- [31] Rajput K, Schram M, Blokland W, Alanazi Y, Ramuhalli P, Zhukov A, Peters C and Vilalta R 2024 *Mach. Learn.: Sci. Technol.* **5** 015044
- [32] Alanazi Y, Schram M, Rajput K, Goldenberg S, Vidyaratne L, Pappas C, Radaideh M I, Lu D, Ramuhalli P and Cousineau S 2023 *Mach. Learn. Appl.* **13** 100484
- [33] Yang Z, Wang J and Zhu Y 2022 Few-shot classification with contrastive learning *Computer Vision - ECCV 2022: 17th European Conf., (Tel Aviv, Israel, 23 October–27 October, 2022, Proc., Part XX)* (Springer) pp 293–309
- [34] Chicco D 2021 *Siamese Neural Networks: An Overview* (*Artificial Neural Networks*) ed H Cartwright (Springer pp 73–94
- [35] Suetsugu Y 2024 *Phys. Rev. Accel. Beams* **27** 063201
- [36] Schmidt R M 2019 Recurrent neural networks (RNNs): a gentle introduction and overview (arXiv:1912.05911)
- [37] Solopova A et al 2019 SRF cavity fault classification using machine learning at CEBAF *Proc. 10th Int. Particle Accelerator Conf. (IPAC'19), (Melbourne, Australia, 19 May–24 May 2019)* (*Int. Particle Accelerator Conf.*) (JACoW Publishing) pp 1167–70
- [38] Pappas G, Lu D, Schram M and Vrabie D 2021 Machine learning for improved availability of the SNS Klystron high voltage converter modulators *Proc. IPAC'21 (Int. Particle Accelerator Conf.)* (JACoW Publishing) pp 4303–6
- [39] Roussel R, Gonzalez-Aguilera J P, Kim Y-K, Wisniewski E, Liu W, Piot P, Power J, Hanuka A and Edelen A 2021 *Nat. Commun.* **12** 3
- [40] Hellert T, Ford T, Leemann S C, Nishimura H, Venturini M and Pollastro A 2024 *Phys. Rev. Accel. Beams* **27** 074602
- [41] Kirkpatrick J et al 2017 *Proc. Natl Acad. Sci.* **114** 3521–6
- [42] Rasmussen C E and Williams C K I 2006 *Gaussian Processes for Machine Learning* vol 103 (MIT Press)
- [43] Duris J et al 2020 *Phys. Rev. Lett.* **124** 124801
- [44] Xu C, Boltz T, Mochihashi A, Santamaria Garcia A, Schuh M and Müller A-S 2023 *Phys. Rev. Accel. Beams* **26** 034601
- [45] Awal A, Hetzel J, Gebel R, Kamerdzhev V and Pretz J 2023 *J. Instrum.* **18** 04010
- [46] Miskovich S A et al 2022 *Phys. Rev. Accel. Beams* **25** 044601
- [47] Morita Y, Washio T and Nakashima Y 2023 *Nucl. Instrum. Methods Phys. Res. A* **1057** 168730
- [48] Lübsen J O, Schütte M, Schulz S and Eichler A 2023 *IFAC-PapersOnLine* **56** 3079–85
- [49] Agapov I, Geloni G, Tomin S and Zagorodnov I 2017 Automatic tuning of free electron lasers (arXiv:1704.02335)

- [50] Xu C, Bründermann E, Müller A S and Garcia A S 2023 Bayesian optimization for sase tuning at the European xfel *Proc. IPAC'23 14th Int. Particle Accelerator Conf.* (JACoW Publishing) pp 4432–5
- [51] Irshad F, Karsch S and Döpp A 2023 *Phys. Rev. Res.* **5** 013063
- [52] Irshad F, Eberle C, Foerster F M, Grafenstein K v, Haberstroh F, Travac E, Weisse N, Karsch S and Döpp A 2024 *Phys. Rev. Lett.* **133** 085001
- [53] Hanuka A, Huang X, Shtalenkova J, Kennedy D, Edelen A, Zhang Z, Lalchand V, Ratner D and Duris J 2021 *Phys. Rev. Accel. Beams* **24** 072802
- [54] Tang C, Abbatematteo B, Hu J, Chandra R, Martín-Martín R and Stone P 2025 *Ann. Rev. Control Robot. Auto. Syst.* **8** 153–88
- [55] Rajput K 2023 Multi-objective optimization of cebaf heat load management and trip rates using AI/ML *Technical Report* Thomas Jefferson National Accelerator Facility (TJNAF)
- [56] Awal A, Hetzel J, Gebel R and Pretz J 2025 *Phys. Rev. Accel. Beams* **28** 034601
- [57] Hirlander S and Bruchon N 2022 Model-free and Bayesian ensembling model-based deep reinforcement learning for particle accelerator control demonstrated on the Fermi fel (arXiv:2012.09737)
- [58] Pang X, Thulasidasan S and Rybarczyk L 2020 Autonomous control of a particle accelerator using deep reinforcement learning (arXiv:2010.08141)
- [59] Schenk M, Combarro E F, Grossi M, Kain V, Li K S B, Popa M-M and Vallecorsa S 2024 *Quantum Sci. Technol.* **9** 025012
- [60] Kain V, Hirlander S, Goddard B, Velotti F M, Della Porta G Z, Bruchon N and Valentino G 2020 *Phys. Rev. Accel. Beams* **23** 124801
- [61] Xiang-Ping Q, Ze-En Y and Qiang W 2014 *Nucl. Sci. Tech.* **25** 203
- [62] Hirlander S, Appel S and Madysa N 2024 Data-driven model predictive control for automated optimization of injection into the sis18 synchrotron: data-driven model predictive control for automated optimization of injection into the sis18 synchrotron *15th Int. Particle Accelerator Conf. (19 May–24 May)*
- [63] Blanco E, de Prada C, Cristea S and Casas J 2009 *Control Eng. Pract.* **17** 1136–47
- [64] Kong Y, Hur M G, Lee E, Park J, Park Y and Yang S 2016 *Nucl. Instrum. Methods Phys. Res. A* **806** 55–60
- [65] Edelen A, Biedron S, Milton S, Bowring D, Chase B, Edelen J, Nicklaus D and Steimel J 2016 (available at: <https://accelconf.web.cern.ch/napac2016/papers/mopob17.pdf>)
- [66] Tao F, Xiao B, Qi Q, Cheng J and Ji P 2022 *J. Manuf. Syst.* **64** 372–89
- [67] der Geer V and de Mj Marieke L 2001 The general particle tracer code: design, implementation and application (available at: <https://api.semanticscholar.org/CorpusID:141078611>)
- [68] Yan Y T, Cai Y, Decker F J, Ecklund S, Irwin J, Seeman J, Sullivan M, Turner J and Wienands U 2005 Virtual accelerator for accelerator optics improvement *Particle Accelerator Conf. (Knoxville, TN, 16–20 May 2005)* vol 0505161 pp 3426
- [69] Sagan D 2006 *Nucl. Instrum. Methods A* **558** 356–9
- [70] Shishlo A, Cousineau S, Holmes J and Gorlov T 2015 *Proc. Comput. Sci.* **51** 1272–81
- [71] Adelmann A 2019 *SIAM/ASA J. Uncertain. Quantif.* **7** 383–416
- [72] Kafkes D and Schram M 2021 Developing robust digital twins and reinforcement learning for accelerator control systems at the Fermilab booster *12th Int. Particle Accelerator Conf.*
- [73] Saoulis A, Baker K, Basak S, Cavanagh H, Cha J, Thiayalingam J and Williamson R 2022 *JACoW IPAC2022 TUOT057*
- [74] Villarreal J, Winklehner D, Koser D and Conrad J M 2024 *Mach. Learn.: Sci. Technol.* **5** 025009
- [75] Sun K, Chen X, Zhao X, Qi X, Wang Z and He Y 2023 *Int. J. Mod. Phys. A* **38** 2350145
- [76] Ögren J, Gohil C and Schulte D 2021 *J. Instrum.* **16** 05012
- [77] Schram M, Rajput K, Li P, St John J and Sharma H 2023 *Phys. Rev. Accel. Beams* **26** 044602
- [78] Zadeh M M, Gethmann J, Müller A S and De Carne G 2024 Development of real-time digital twins for particle accelerators 2024 *Energy Conversion Congress & Expo Europe (ECCE Europe)* pp 1–5
- [79] Edelen A et al 2024 Progress on combining digital twins and machine learning-based control for accelerators at slac (available at: <https://inspirehep.net/literature/2812840>)
- [80] Emma C, Edelen A, Hogan M J, O'Shea B, White G and Yakimenko V 2018 *Phys. Rev. Accel. Beams* **21** 112802
- [81] Sanchez-Gonzalez A et al 2016 *Nat. Commun.* **8** 15461
- [82] Jafarinia F et al 2023 Simulation study on a virtual diagnostics concept for x-ray pulse characterisation *Proc. 14th Int. Particle Accelerator Conf. (IPAC'23 -)* (JACoW Publishing) pp 1813–5
- [83] Schienker A, Edelen A, Bohler D, Emma C and Lutman A 2018 *Phys. Rev. Lett.* **121** 044801.1–6
- [84] Wares S, Isaacs J and Elyan E 2019 *SN Appl. Sci.* **1** 1–19
- [85] Widmer G and Kubat M 1996 *Mach. Learn.* **23** 69–101
- [86] Edelen A and Huang X 2024 *Annu. Rev. Nucl. Part. Sci.* **74** 557–81
- [87] Safranek J, Corbett W, Hettel R, Huang X, Nosochkov Y, Sebek J and Terebilo A 2007 Spear3 accelerator physics update 2007 *IEEE Particle Accelerator Conf. (PAC)* pp 1311–3
- [88] Wendt M 2021 *IEEE Instrum. Meas. Mag.* **24** 21–32
- [89] Blokland W and Peters C C 2013 A new differential and errant beam current monitor for the sns* accelerator *Proc. 2nd Int. Beam Instrumentation Conf. (IBIC'13)* (JACoW Publishing) pp 921–4
- [90] Parisi G I, Kemker R, Part J L, Kanan C and Wermter S 2019 *Neural Netw.* **113** 54–71
- [91] Mermillod M, Bugaiska A and BONIN P 2013 *Front. Psychol.* **4** 504
- [92] Bifet A and Gavaldá R 2007 *Learning From Time-Changing Data With Adaptive Windowing* pp 443–8
- [93] Gama J, Medas P, Castillo G and Rodrigues P 2004 Learning with drift detection *Advances in artificial intelligence – SBIA 2004* ed A L C Bazzan and S Labidi (Springer) pp 286–95
- [94] Yang J, Zhou K, Li Y and Liu Z 2024 *Int. J. Comput. Vis.* **132** 5635–62
- [95] Fedus W, Ramachandran P, Agarwal R, Bengio Y, Larochele H, Rowland M and Dabney W 2020 Revisiting fundamentals of experience replay *Proc. 37th Int. Conf. on Machine Learning (Proc. Machine Learning Research)* vol 119, ed A Singh (PMLR) pp 3061–71
- [96] Rolnick D, Ahuja A, Schwarz J, Lillicrap T and Wayne G 2019 Experience replay for continual learning *Advances in Neural Information Processing Systems* vol 32, ed H Wallach, H Larochele, A Beygelzimer, F d' Alché-Buc, E Fox and R Garnett (Curran Associates, Inc.)
- [97] Schaul T, Quan J, Antonoglou I and Silver D 2016 Prioritized experience replay (arXiv:1511.05952)
- [98] Shin H, Lee J K and Kim J 2017 Continual learning with deep generative replay *Advances in Neural Information Processing Systems* vol 30, ed I Guyon, U V Luxburg, S Bengio, H Wallach, R Fergus, S Vishwanathan and R Garnett (Curran Associates, Inc.)

- [99] Riemer M, Cases I, Ajemian R, Liu M, Rish I, Tu Y and Tesauro G 2019 Learning to learn without forgetting by maximizing transfer and minimizing interference *Int. Conf. on Learning Representations*
- [100] Rusu A A, Rabinowitz N C, Desjardins G, Soyer H, Kirkpatrick J, Kavukcuoglu K, Pascanu R and Hadsell R 2022 Progressive neural networks (arXiv:1606.04671)
- [101] Schmidhuber J 1992 *Neural Comput.* **4** 131–9
- [102] Zhang P, Yan Y, Li C, Wang S, Xie X, Song G and Kim S 2023 Continual learning on dynamic graphs via parameter isolation *Proc. 46th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval (ACM)* pp 601–11
- [103] Aljundi R et al 2018 Memory aware synapses: learning what (not) to forget *Proc. 35th Int. Conf. on Machine Learning* vol 80 pp 2490–9
- [104] Li Z and Hoiem D 2018 *IEEE Trans. Pattern Anal. Mach. Intell.* **40** 2935–47
- [105] Hinton G, Vinyals O and Dean J 2015 Distilling the knowledge in a neural network *NIPS Deep Learning and Representation Learning Workshop*
- [106] Lopez-Paz D and Ranzato M A 2017 Gradient episodic memory for continual learning *Advances in Neural Information Processing Systems* vol 30, ed I Guyon, U V Luxburg, S Bengio, H Wallach, R Fergus, S Vishwanathan and R Garnett (Curran Associates, Inc.)
- [107] Farajtabar M, Azizan N, Mott A and Li A 2020 Orthogonal gradient descent for continual learning *Proc. 23rd Int. Conf. on Artificial Intelligence and Statistics (Proc. Machine Learning Research)* vol 108, ed S Chiappa and R Calandra (PMLR) pp 3762–73
- [108] Oza N C and Russell S J 2001 Online bagging and boosting *Proc. 8th Int. Workshop on Artificial Intelligence and Statistics (Proc. Machine Learning Research)* vol R3, ed T S Richardson and T S Jaakkola (PMLR) pp 229–36
- [109] Rosenbaum C, Klinger T and Riemer M 2018 Routing networks: adaptive selection of non-linear functions for multi-task learning *Int. Conf. on Learning Representations*
- [110] Vinyals O, Blundell C, Lillicrap T, kavukcuoglu k and Wierstra D 2016 Matching networks for one shot learning *Advances in Neural Information Processing Systems* vol 29, ed D Lee, M Sugiyama, U Luxburg, I Guyon and R Garnett (Curran Associates, Inc.)
- [111] Finn C, Rajeswaran A, Kakade S and Levine S 2019 Online meta-learning *Int. Conference on Machine Learning (PMLR)* pp 1920–30
- [112] Aljundi R, Lin M, Goujaud B and Bengio Y 2019 *Advances in Neural Information Processing Systems* vol 32
- [113] Chaudhry A, Dokania P K, Ajanthan T and Torr P H S 2018 *Riemannian Walk for Incremental Learning: Understanding Forgetting and Intransigence* (Springer) pp 556–72
- [114] Kubernetes Documentation 2024 Getting started — kubernetes.io (available at: <https://kubernetes.io/docs/setup/>) (Accessed 17 February 2025)
- [115] Bard D J, Day M R, Enders B, Hartman-Baker R J, Riney J, Snavely C and Torok G 2021 Automation for data-driven research with the nersc superfacility API *High Performance Computing* ed H Jagode, H Anzt, H Ltaief and P Luszczek (Springer) pp 333–45
- [116] Chen A et al 2020 Developments in mlflow: a system to accelerate the machine learning lifecycle *Proc. 4th Int. Workshop on Data Management for End-to-End Machine Learning DEEM'20* (Association for Computing Machinery)
- [117] Pythonic 2024 Modern workflow orchestration for resilient data platforms | prefect — (available at: www.prefect.io/) (Accessed 17 February 2025)
- [118] Yasmin J, Wang J A, Tian Y and Adams B 2025 *J. Syst. Softw.* **219** 112248
- [119] Andrychowicz M, Wolski F, Ray A, Schneider J, Fong R, Welinder P, McGrew B, Tobin J, Abbeel P and Zaremba W 2017 Hindsight experience replay *Proc. 31st Int. Conf. on Neural Information Processing Systems NIPS'17* (Curran Associates Inc.) pp 5055–65
- [120] Nagabandi A, Clavera I, Liu S, Fearing R S, Abbeel P, Levine S and Finn C 2019 Learning to adapt in dynamic, real-world environments through meta-reinforcement learning (arXiv:1803.11347)
- [121] Massey F J 1951 *J. Am. Stat. Assoc.* **46** 68–78
- [122] Pearson K 1900 *London, Edinburgh Dublin Phil. Mag. J. Sci.* **50** 157–75
- [123] Villani C 2009 *The Wasserstein Distances* (Springer) pp 93–111
- [124] Menendez M, Pardo J, Pardo L and Pardo M 1997 *J. Franklin Inst.* **334** 307–18
- [125] Joyce J M 2011 *Kullback-Leibler Divergence* (Springer) pp 720–2
- [126] Baena-García M, Campo-Ávila J, Fidalgo-Merino R, Bifet A, Gavalda R and Morales-Bueno R 2006 (available at: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=2747577a61c70bc3874380130615e15aff76339e>)
- [127] Pesaranghader A, Viktor H L and Paquet E 2018 Mcdiarmid drift detection methods for evolving data streams *2018 Int. Joint Conf. on Neural Networks (IJCNN)* pp 1–9
- [128] Goan E and Fookes C 2020 *Bayesian Neural Networks: An Introduction and Survey* (Springer) pp 45–87
- [129] Liu J Z, Lin Z, Padhy S, Tran D, Bedrax-Weiss T and Lakshminarayanan B 2020 Simple and principled uncertainty estimation with deterministic deep learning via distance awareness *Proc. 34th Int. Conf. on Neural Information Processing Systems (NIPS'20)* (Curran Associates Inc.)
- [130] Chen Y, Sung C L, Kusari A, Song X and Sun W 2024 Uncertainty-aware out-of-distribution detection with Gaussian processes (arXiv:2412.20918)
- [131] Britton T, Goodrich M, Jarvis N, Jeske T, Kalra N, Lawrence D, McSpadden D and Rajput K 2024 *J. Instrum.* **19** C11012
- [132] Mnih V et al 2015 *Nature* **518** 529–33
- [133] Luo Y, Wang Y, Dong K, Zhang Q, Cheng E, Sun Z and Song B 2023 *Neurocomputing* **557** 126620
- [134] van de Ven G M and Tolias A S 2019 Three scenarios for continual learning (arXiv:1904.07734)
- [135] Hayes T L, Cahill N D and Kanan C 2019 Memory efficient experience replay for streaming learning *2019 Int. Conf. on Robotics and Automation (ICRA)* pp 9769–76
- [136] Sun L 2023 Robot control policy transfer based on progressive neural network *2023 IEEE 3rd Int. Conf. on Information Technology, Big Data and Artificial Intelligence (ICIBA)* vol 3 pp 1285–9
- [137] Poth C, Sterz H, Paul I, Purkayastha S, Engländer L, Imhof T, Vulić I, Ruder S, Gurevych I and Pfeiffer J 2023 Adapters: a unified library for parameter-efficient and modular transfer learning *Proc. 2023 Conf. on Empirical Methods in Natural Language Processing: System Demonstrations* pp 149–60
- [138] Qiao J, Zhang Z, Tan X, Qu Y, Zhang W, Han Z and Xie Y 2024 Gradient projection for continual parameter-efficient tuning (arXiv:2405.13383)
- [139] Chaudhry A, Ranzato M, Rohrbach M and Elhoseiny M 2019 *ICLR 2019 Proc.*
- [140] Saha G, Garg I and Roy K 2021 Gradient projection memory for continual learning *Int. Conf. on Learning Representations*
- [141] Lin S, Yang L, Fan D and Zhang J 2022 *10th Int. Conf. on Learning Representations, ICLR 2022*
- [142] Ali A, Hamraz M, Gul N, Khan D M, Aldahmani S and Khan Z 2023 *Pattern Recognit.* **142** 109641
- [143] Kuncheva L 2004 *Bagging and Boosting* (Wiley) ch 7, pp 203–35

- [144] Hospedales T M, Antoniou A, Micaelli P and Storkey A J 2020 *IEEE Trans. Pattern Anal. Mach. Intell.* **44** 5149–69
- [145] Finn C, Abbeel P and Levine S 2017 Model-agnostic meta-learning for fast adaptation of deep networks *Proc. 34th Int. Conf. on Machine Learning (Proc. Machine Learning Research)* vol 70, ed D Precup and Y W Teh (PMLR) pp 1126–35
- [146] Andrychowicz M, Denil M, Gómez S, Hoffman M W, Pfau D, Schaul T, Shillingford B and de Freitas N 2016 Learning to learn by gradient descent by gradient descent *Advances in Neural Information Processing Systems* vol 29, ed D Lee, M Sugiyama, U Luxburg, I Guyon and R Garnett (Curran Associates, Inc.)
- [147] Snell J, Swersky K and Zemel R 2017 Prototypical networks for few-shot learning *Advances in Neural Information Processing Systems* vol 30, ed I Guyon, U V Luxburg, S Bengio, H Wallach, R Fergus, S Vishwanathan and R Garnett (Curran Associates, Inc.)
- [148] Ravi S and Larochelle H 2017 Optimization as a model for few-shot learning *Int. Conf. on Learning Representations*
- [149] Lake B M, Salakhutdinov R R and Tenenbaum J 2013 One-shot learning by inverting a compositional causal process *Advances in Neural Information Processing Systems* vol 26, ed C Burges, L Bottou, M Welling, Z Ghahramani and K Weinberger (Curran Associates, Inc.)
- [150] Xu Z, van Hasselt H P, Hessel M, Oh J, Singh S and Silver D 2020 Meta-gradient reinforcement learning with an objective discovered online *Advances in Neural Information Processing Systems* vol 33, ed H Larochelle, M Ranzato, R Hadsell, M Balcan and H Lin (Curran Associates, Inc.) pp 15254–64
- [151] Hirilaender S, Pochaba S, Lamminger L, Kaiser J, Xu C, Santamaria Garcia A and Scomparin L 2024 Towards few-shot reinforcement learning in particle accelerator control [Ebook] *15th Int. Particle Accelerator Conf., (Nashville, Tennessee: 19 May–24 May, 2024)* (JACoW Publishing) pp 1804–7
- [152] Sow D, Lin S, Liang Y and Zhang J 2024 Algorithm design for online meta-learning with task boundary detection *Conf. on Parsimony and Learning* (PMLR) pp 458–79